OPUS

User Manual

==========

OPUS v0.9

Forensic, Steganographic & Cryptanalytic Toolkit

==========

Unified User + Technical Manual

Author: Gregory Novak

Release: Internal Engineering Draft

==========

==========

CHAPTER 1 — PREFACE

Opus Unified Manual (User + Technical Edition)

==========

Opus is a modular, extensible, command-line forensic suite designed for analysts, CTF competitors, reverse engineers, and digital investigators who require a tool that is both powerful and transparent. This manual represents the **complete**, **exhaustive**, and **painstakingly detailed** documentation of Opus — a unified text that merges user-facing explanations with deep technical insight, subsystem rationale, and extended examples.

This is the **Opus Codex** — a comprehensive reference intended to be read slowly, absorbed gradually, and consulted repeatedly. It is intentionally verbose, intentionally methodical, and intentionally structured to provide clarity even at the cost of length. It is the kind of manual that rewards patience, curiosity, and a willingness to explore the deeper layers of a forensic toolkit.

Preface

Opus was created to solve a problem that many analysts encounter but few tools address cleanly: the need for a single, coherent, modular environment capable of handling the messy, layered, unpredictable nature of real-world forensic data. In the field — whether that field is a CTF arena, a malware lab, or a digital forensics workstation — files rarely present themselves in neat, isolated forms. Instead, they arrive as archives within archives, images hiding payloads, binaries with embedded structures, ciphertexts with misleading patterns, and compressed blobs that defy casual inspection.

Opus was built to confront this reality directly.

It is a toolkit designed not merely to run commands, but to guide the analyst through a structured process of discovery, peeling back layers of complexity one step at a time. It is built around the idea that forensic analysis is not a single action but a pipeline, a sequence of transformations and observations that gradually reveal the truth hidden within data.

This manual reflects that philosophy. It is structured to mirror the way Opus itself operates: modular, layered, recursive, and thorough. Each chapter builds upon the last, providing both conceptual understanding and practical application. The goal is not only to teach you how to use Opus, but to help you understand why Opus behaves the way it does — why certain design decisions were made, why certain safeguards exist, and why certain workflows are recommended.

Throughout this manual, you will encounter:

- long-form explanations that break down complex ideas
- extended examples that show Opus in action
- technical notes that clarify internal behavior
- warnings and caveats that reflect real-world experience
- ASCII diagrams that illustrate workflows
- cross-references to help you navigate the text

This is a manual designed to be read linearly, but also one that supports nonlinear exploration. If you wish to jump directly to the extraction engine, you may. If you want to study the cryptanalysis subsystem in isolation, you can. If you want to understand the philosophy behind Opus's safety model, that too is available.

The manual is intentionally dense, verbose, and comprehensive — the kind of document that a developer might jokingly describe as "too much information," but which an analyst will recognize as a valuable resource. It is meant to be a companion to Opus itself, a reference that grows with the toolkit and evolves as new features are added.

Intended Audience

Opus is designed for practitioners and learners across the forensic and cryptanalytic spectrum, including:

- **forensic analysts**
- **CTF competitors**
- **reverse engineers**
- **security researchers**
- **digital investigators**
- **cryptanalysis enthusiasts**
- **developers integrating Opus into workflows**

The manual assumes working familiarity with:

- **command-line environments**
- **basic cryptographic concepts**
- **file formats and signatures**
- **compression and encoding**
- **digital forensics principles**

Expert-level knowledge is *not* required. Each subsystem is introduced with clear explanations, practical examples, and the rationale behind design decisions. The goal is to make Opus accessible to newcomers while still offering the depth and precision expected by experienced analysts.

Document Conventions

To ensure clarity and consistency, this manual uses the following conventions:

Monospaced Blocks

Used for commands, file paths, and output examples.

Bold Terms

Used for important concepts and subsystem names.

Notes

Used to highlight subtle behaviors or caveats.

Warnings

Used to emphasize potential pitfalls or dangerous operations.

ASCII Diagrams

Used to illustrate workflows and architecture.

Inline Exploration Highlights

Used to mark concepts you may want to explore further, such as **recursive extraction behavior** or **entropy interpretation**.

Philosophy of Opus Documentation

Opus is a toolkit for people who think deeply, investigate thoroughly, and rely on precision. Its documentation reflects that same mindset.

This manual is intentionally long. It is intentionally detailed. It is intentionally exhaustive.

The goal is not brevity — the goal is clarity.

Opus is designed for analysts who need to understand what their tools are doing, why they are doing it, and how to interpret the results with confidence. That requires documentation that goes beyond surface-level descriptions. It requires explanations that illuminate the internal logic of each subsystem, the assumptions behind each algorithm, and the reasoning that shaped each design choice.

If you ever find yourself thinking, *"This is more detail than I expected,"* then the manual is doing exactly what it was meant to do.

Documentation Principles

Transparency

Every subsystem in Opus is documented with the same guiding principle: nothing should feel like a black box. Analysts deserve to know how a tool reaches its conclusions. This manual explains the mechanics, the heuristics, and the edge cases so you can trust — and verify — the output.

Reproducibility

Forensic work demands repeatability. Every example in this manual is crafted so you can reproduce it exactly, understand the conditions under which it works, and adapt it to your own environment. When Opus performs an analysis, you should be able to follow the same steps manually if needed.

Rationale-Driven Design

Opus is not a random collection of utilities. Each subsystem exists for a reason, and each feature is shaped by real investigative needs. This manual explains not only *how* to use a feature, but *why* it behaves the way it does — the tradeoffs, the constraints, and the analytical philosophy behind it.

Depth Over Convenience

Many tools prioritize short descriptions and quick usage examples. Opus takes a different approach. Analysts often need to understand the deeper structure of a file format, the nuances of an encoding scheme, or the implications of a cryptographic transformation. This manual provides that depth so you can make informed decisions, not guesses.

Contextual Learning

Every chapter is structured to teach you not just the command syntax, but the surrounding concepts. When Opus analyzes a PNG, you'll learn how PNG chunks work. When Opus detects steganography, you'll learn the underlying patterns. When Opus decodes a cipher, you'll learn the cryptographic assumptions.

The manual is a reference, but it is also a learning tool.

Why This Level of Detail Matters

Digital forensics, CTF challenges, and cryptanalysis all share a common truth: the smallest detail can change the entire interpretation of a result. A single byte offset, a misaligned header, a misunderstood encoding — these can be the difference between a solved case and a dead end.

Opus is built to help you catch those details. This manual is built to help you understand them.

By documenting every subsystem thoroughly, Opus ensures that:

- you know what the tool is doing internally
- you can validate or challenge its conclusions
- you can adapt its logic to new or unexpected data
- you can integrate it into larger workflows with confidence

This is not “press a button and hope for the best” software. This is a toolkit for people who want to know what’s happening under the hood.

A Manual That Grows With You

Whether you’re a newcomer learning the fundamentals or an experienced analyst pushing Opus into new territory, the documentation is structured to meet you where you are. Each chapter begins with approachable explanations and builds toward deeper, more technical insights. You can skim when you’re in a hurry, or you can dive deep when you want mastery.

Opus is a living project, and this manual evolves with it. As new subsystems are added, the same philosophy applies: clarity, transparency, reproducibility, and depth.

==========

CHAPTER 2 — INTRODUCTION TO OPUS

==========

Opus is not merely a collection of forensic utilities. It is not a loose assortment of scripts, nor a bundle of unrelated tools stitched together for convenience. Instead, Opus is a **cohesive forensic environment**, a unified system built around the idea that digital artifacts rarely exist in isolation and that meaningful analysis requires a toolkit capable of navigating complexity with precision, consistency, and depth. This chapter introduces Opus from a conceptual standpoint, explaining what it is, what it is not, and why it was created. It also provides a high-level overview of its capabilities, its design philosophy, and the analytical mindset it encourages.

2.1 What Opus Is

Opus is a **modular, extensible, command-line forensic suite** designed to support analysts working with complex, layered, or obfuscated data. It is built to handle the kinds of files and challenges that arise in CTF competitions, digital forensics investigations, malware analysis, and reverse-engineering workflows. At its core, Opus is a system that embraces **recursion**, **automation**, and **structured analysis**, enabling users to peel back layers of data in a controlled and repeatable manner.

Opus is designed to be a **single environment** where an analyst can:

- open files
- inspect their structure
- extract embedded content
- analyze entropy
- detect steganographic artifacts
- perform cryptanalysis
- inspect binaries
- carve files from raw data
- and recursively process complex archives

This unified approach eliminates the need to switch between multiple tools, reducing cognitive load and ensuring that analysis remains consistent across different file types and workflows. Opus is built to be a **trusted companion** in the investigative process — a tool that behaves predictably, reports clearly, and provides analysts with the information they need to make informed decisions.

Opus is also designed to be **transparent**. It does not hide its operations behind opaque interfaces or silent heuristics. Instead, it prints detailed messages explaining what it is doing, why it is doing it, and what the results mean. This transparency is essential for forensic work, where analysts must be able to justify their conclusions and understand the provenance of every piece of data they examine. This commitment to clarity is reflected throughout the manual, especially in sections like **recursive extraction behavior** and **entropy interpretation**.

2.2 What Opus Is Not

Opus is not a GUI-based forensic suite. It does not attempt to replicate the functionality of large commercial platforms, nor does it aim to provide a point-and-click interface for casual users. Instead, Opus embraces the command line as a powerful, flexible, and expressive environment for forensic analysis. This choice reflects the belief that serious analysts benefit from tools that prioritize control, precision, and scriptability over visual convenience.

Opus is also not a replacement for specialized tools such as full-scale disassemblers, memory forensics frameworks, or network analysis suites. While it includes binary inspection tools, cryptanalysis engines, and steganographic analyzers, it is not intended to replace dedicated platforms like Ghidra, Volatility, or Wireshark. Instead, Opus complements these tools by providing a **focused, modular environment** for file-centric analysis — the kind of work that often precedes or accompanies deeper investigation.

Finally, Opus is not a black-box solver. It does not magically "solve" challenges without user input or understanding. Instead, it provides **structured assistance**, guiding the analyst through processes such as Vigenère cracking, substitution solving, entropy interpretation, and JPEG steganalysis. The goal is not to automate thinking, but to **augment it**, giving analysts the tools they need to reason effectively about complex data.

2.3 Why Opus Exists

Opus was created to address a gap in the forensic tooling landscape: the lack of a **unified, modular, command-line environment** capable of handling the messy, layered nature of real-world data. Many existing tools are either too specialized, too opaque, or too fragmented to support the kind of recursive, multi-stage analysis that modern challenges demand.

In CTF competitions, for example, analysts frequently encounter files that contain:

- compressed archives
- inside steganographic images
- inside encrypted containers
- inside binary payloads
- inside corrupted or partially overwritten structures

Traditional tools often require analysts to manually extract each layer, switch between utilities, and track intermediate files. This process is error-prone, time-consuming, and mentally taxing.

Opus was designed to solve this problem by providing a **single, coherent workflow** for recursive extraction and analysis. Its extraction engine, carving subsystem, entropy analyzer, and string intelligence modules work together to reveal hidden layers of data with minimal manual intervention. This design philosophy is explored in depth in later chapters, particularly in the sections on **recursive extraction** and **file carving**.

Opus also exists to provide a **transparent, educational environment** for analysts who want to understand the inner workings of forensic processes. By printing detailed messages, exposing intermediate results, and offering clear explanations, Opus helps users develop intuition about file structures, compression formats, entropy patterns, and cryptographic weaknesses.

2.4 High-Level Overview of Capabilities

Opus includes a wide range of capabilities, each designed to support a specific aspect of forensic analysis. These capabilities are organized into subsystems, which are described in detail in later chapters. At a high level, Opus provides:

File Operations

Opening, copying, hashing, deleting, and recursively extracting files.

Forensic Analysis

Entropy analysis, string extraction, magic byte detection, and embedded file discovery.

Steganography Tools

JPEG LSB analysis, chi-square testing, RS analysis, DCT profiling, and Huffman fingerprinting.

Cryptanalysis Tools

Vigenère solvers, substitution solvers, quadgram scoring, RSA factorization, and designation analysis.

Binary Inspection Tools

ELF inspection, PIE base calculation, and return-address timing estimation.

Recursive Extraction Engine

A multi-stage pipeline that unwraps compressed formats, carves embedded files, and analyzes each layer.

String Intelligence

Detection of ASCII, UTF-8, base64, URLs, and emails, with offset reporting.

Thread Pool

Background execution of long-running tasks with live status updates.

Each of these capabilities is explored in depth in later chapters, with examples, explanations, and detailed workflows.

2.5 The Analytical Mindset Encouraged by Opus

Opus is designed to encourage a particular way of thinking — a mindset that values **curiosity**, **patience**, **methodical exploration**, and **structured reasoning**. Forensic analysis is rarely a linear

process; it often involves branching paths, unexpected discoveries, and iterative refinement. Opus supports this mindset by providing tools that are flexible, transparent, and composable.

When using Opus, analysts are encouraged to:

- explore data recursively
- examine entropy patterns
- inspect strings and offsets
- analyze file signatures
- test steganographic hypotheses
- attempt cryptanalysis iteratively
- interpret results critically

This mindset is reflected throughout the manual, especially in sections like **multi-layer extraction workflows** and **cryptanalysis case studies**.

2.6 How Opus Fits Into the Forensic Landscape

Opus occupies a unique position in the forensic ecosystem. It is not a replacement for large, GUI-based platforms, nor is it a simple collection of scripts. Instead, it is a **bridge** — a tool that connects the flexibility of the command line with the structure of a forensic suite.

Analysts may use Opus:

- before loading a file into a disassembler
- after extracting data from a disk image
- during CTF challenges
- while triaging suspicious files
- when investigating stego artifacts
- when analyzing encrypted or encoded data

Opus excels in situations where data is **layered**, **obfuscated**, or **unfamiliar**, providing a structured environment for peeling back complexity.

==========

CHAPTER 3 — ARCHITECTURE OVERVIEW

==========

Opus is built on a modular, layered architecture designed to support complex forensic workflows with clarity, consistency, and reliability. This chapter provides a comprehensive overview of the internal structure of Opus, explaining how its subsystems interact, how data flows through the toolkit, and how the design choices support both usability and analytical rigor. While Opus is implemented in C, this chapter avoids code-level detail and instead focuses on the conceptual architecture — the high-level structures, responsibilities, and interactions that define how Opus behaves. The goal is to give analysts a deep understanding of the system's internal logic without requiring them to read or modify the source code.

3.1 Architectural Philosophy

The architecture of Opus is guided by several core principles that shape every subsystem and workflow. These principles are not merely abstract ideals; they are practical constraints that ensure Opus remains reliable, predictable, and effective in real-world forensic scenarios.

Modularity

Each subsystem in Opus is designed to operate independently, with clearly defined responsibilities and minimal coupling. This modularity allows analysts to use only the components they need while ensuring that each subsystem can be extended or improved without affecting unrelated parts of the toolkit. This principle is especially important in areas like **recursive extraction behavior** and **cryptanalysis workflows**, where complexity can easily lead to tangled dependencies if not carefully managed.

Transparency

Opus is designed to be transparent in its operations. Every significant action — from file detection to entropy calculation to solver restarts — is printed to the screen in clear, readable messages. This transparency ensures that analysts always understand what Opus is doing and why, which is essential for forensic integrity and reproducibility.

Safety

Forensic tools must be safe by default. Opus enforces strict safeguards to prevent accidental data loss, infinite recursion, or unsafe file operations. These safeguards are woven into the architecture at every level, from the extraction engine to the thread pool. The safety model is explored in depth in later chapters, particularly in the section on **sandboxing and controlled file I/O**.

Recursion as a First-Class Concept

Many forensic challenges involve layered or nested data structures. Opus treats recursion not as an afterthought but as a core architectural feature. The extraction engine, file carver, and analysis modules are all designed to operate recursively, allowing Opus to peel back layers of complexity automatically and systematically.

Extensibility

Opus is built to grow. New subsystems, commands, and analysis modules can be added without disrupting existing functionality. This extensibility is supported by a clean separation of concerns and a consistent interface design.

3.2 High-Level System Structure

At a high level, Opus is composed of four major architectural layers:

1. **User Interface Layer**
2. **Core Engine Layer**
3. **Forensic & Cryptanalytic Modules**
4. **Support Systems**

Each layer has a distinct role, and together they form a cohesive forensic environment.

3.3 User Interface Layer

The user interface layer is responsible for all interactions between the analyst and the toolkit. It includes the menu system, command parser, status line renderer, and output formatting utilities. While Opus is a command-line tool, the interface layer is designed to be intuitive, discoverable, and consistent.

Menu System

The menu system provides a structured overview of available commands, grouped by category. It serves as both a navigation tool and a reference guide, helping analysts explore Opus's capabilities without memorizing every command.

Command Parser

The command parser interprets user input, validates arguments, and dispatches commands to the appropriate subsystems. It handles edge cases, error messages, and ambiguous input gracefully, ensuring that analysts receive clear feedback when commands are misused.

Status Line Renderer

The status line provides real-time updates on background tasks, particularly during long-running cryptanalysis operations. It displays information such as the number of active solver threads, the current restart count, and the best score achieved so far. This feature is especially useful during **quadgram hillclimbing** and **Vigenère refinement**.

3.4 Core Engine Layer

The core engine layer contains the foundational subsystems that power Opus's most important features. These subsystems include the extraction engine, file carver, entropy engine, and string intelligence module. Each subsystem is designed to operate independently but can be combined to form powerful recursive workflows.

Extraction Engine

The extraction engine is responsible for detecting compressed formats, decompressing files, normalizing suffixes, and recursively analyzing extracted content. It is one of the most complex and powerful components of Opus, capable of handling multi-layer archives and deeply nested structures. The extraction engine is explored in detail in later chapters, particularly in the section on **multi-stage extraction pipelines**.

File Carver

The file carver scans raw byte streams for known file signatures and extracts embedded files. It supports formats such as JPEG, PNG, ZIP, ELF, and PE. The carver is designed to operate both independently and as part of the extraction engine, allowing Opus to uncover hidden content even when file structures are corrupted or incomplete.

Entropy Engine

The entropy engine calculates global entropy, block-level entropy, and LSB distributions. These metrics help analysts identify compressed, encrypted, or steganographically modified data. Entropy analysis is a critical part of forensic workflows, and Opus provides detailed reports that support **entropy interpretation**.

String Intelligence

The string intelligence module extracts ASCII, UTF-8, base64, URLs, and emails from raw data. It also reports offsets and summary statistics, helping analysts identify meaningful content within large or unstructured files.

3.5 Forensic & Cryptanalytic Modules

Opus includes a wide range of specialized modules designed to support forensic and cryptanalytic workflows. These modules operate on top of the core engine layer and provide advanced analysis capabilities.

JPEG Stego Engine

The JPEG stego engine performs LSB analysis, chi-square testing, RS analysis, DCT profiling, and Huffman fingerprinting. It is designed to detect subtle anomalies in JPEG images that may indicate hidden data. This subsystem is explored in depth in the section on **JPEG steganalysis**.

Cryptanalysis Engine

The cryptanalysis engine includes tools for solving Vigenère ciphers, substitution ciphers, and RSA challenges. It uses quadgram scoring, hillclimbing, IC analysis, and factorization algorithms to support a wide range of cryptanalytic tasks. These tools are essential for CTF challenges and are explored in detail in the section on **cryptanalysis workflows**.

Binary Inspection Tools

Opus includes tools for inspecting ELF binaries, calculating PIE bases, and estimating return-address timing. These tools provide valuable insights into binary structures and execution behavior.

Designation Tools

The designation tools support the extraction, decoding, and analysis of Opus designation blocks. These tools are used primarily in specialized workflows and are explored in detail in later chapters.

3.6 Support Systems

The support systems provide essential infrastructure for the rest of the toolkit. These systems include the thread pool, logging utilities, and file I/O wrappers.

Thread Pool

The thread pool manages background tasks, particularly long-running cryptanalysis operations. It ensures that Opus remains responsive even when performing computationally intensive work. The thread pool is designed to be safe, efficient, and transparent, with clear status updates provided through the status line.

Logging

Opus includes logging utilities that record important events, errors, and analysis results. These logs support reproducibility and forensic integrity.

File I/O Utilities

The file I/O utilities provide safe wrappers around file operations, ensuring that Opus never overwrites files, never performs unsafe writes, and always handles errors gracefully. These utilities are essential for maintaining the safety model described in the section on **sandboxing and controlled file operations**.

3.7 Data Flow Through Opus

Data flows through Opus in a structured, predictable manner. The typical workflow begins with a file being opened or loaded, followed by a series of analysis steps that may include extraction, carving, entropy analysis, string extraction, and cryptanalysis. Each subsystem operates independently but can pass results to other subsystems as needed.

A typical data flow might look like this:

1. File is opened
2. Magic byte detection identifies format
3. Extraction engine decompresses content
4. File carver extracts embedded files
5. Entropy engine analyzes data
6. String intelligence extracts meaningful text
7. JPEG stego engine analyzes images
8. Cryptanalysis engine attempts decryption
9. Results are printed and logged

This structured flow ensures that analysts can follow the progression of analysis clearly and confidently.

==========

CHAPTER 4 — INSTALLATION & EXECUTION

==========

Opus is designed to be portable, predictable, and consistent across environments, but like any forensic toolkit implemented in C, it requires a proper build environment and a clear understanding of how its components are compiled, linked, and executed. This chapter provides a comprehensive, deeply detailed walkthrough of the installation and execution process, including compiler requirements, library dependencies, build instructions, troubleshooting strategies, and an explanation of what happens internally when Opus starts. While the process itself is straightforward, this chapter explores it in a level of detail that ensures analysts understand not only *how* to build Opus, but *why* each step matters.

4.1 System Requirements

Opus is designed to run on Unix-like systems, including Linux distributions, BSD variants, and macOS. While it may compile on other platforms, the toolkit assumes a POSIX-compliant environment and relies on system calls, directory semantics, and terminal behavior that are standard on Unix systems.

Minimum Requirements

- A POSIX-compatible operating system
- GCC or Clang
- Standard C library
- Terminal emulator with ANSI support
- At least 512 MB of RAM
- At least 50 MB of free disk space

Recommended Requirements

- GCC 9.0+ or Clang 10.0+
- GNU Make
- libjpeg (for JPEG steganalysis)
- GMP (for RSA operations)
- A terminal with UTF-8 support
- A modern Linux distribution

These requirements ensure that Opus can perform tasks such as **recursive extraction**, **entropy analysis**, and **cryptanalysis workflows** without performance bottlenecks or compatibility issues.

4.2 Dependencies

Opus relies on several external libraries to support its advanced features. These dependencies are chosen for their performance, reliability, and widespread availability.

libjpeg

Used for JPEG parsing, DCT coefficient extraction, and Huffman table analysis. Without libjpeg, the JPEG stego engine cannot operate.

GMP (GNU Multiple Precision Arithmetic Library)

Used for RSA factorization, modular arithmetic, and large integer operations. GMP provides the performance and precision required for **RSA workflows**.

zlib / gzip utilities

Used for decompressing gzip streams during recursive extraction.

Standard C Library

Used for file I/O, memory management, string operations, and system calls.

These dependencies are common on most Unix systems and can be installed via package managers such as apt, yum, pacman, or brew.

4.3 Obtaining the Source Code

Opus is distributed as a source tree containing:

- C source files
- header files
- resource files
- documentation
- build scripts (optional)

The directory structure typically resembles:

```
Opus/
  ├── src/
  ├── include/
  ├── modules/
  ├── diagrams/
```

```
├── docs/
└── Makefile
```

This structure supports modular development and clear separation of concerns, which is essential for maintaining the toolkit's extensibility and clarity.

4.4 Building Opus

Opus is compiled using GCC or Clang. The build process is intentionally simple, reflecting the philosophy that forensic tools should be easy to deploy in diverse environments.

Basic Build Command

```
gcc -Wall -Wextra -o Opus src/*.c -lm -lgmp -ljpeg
```

This command:

- compiles all C source files
- enables warnings for safer code
- links against the math library
- links against GMP
- links against libjpeg

Using Make

If a Makefile is provided, you can build Opus with:

```
make
```

This approach is recommended for environments where dependencies or compiler flags may vary.

4.5 Troubleshooting Build Issues

Even though Opus is designed to build cleanly, analysts may encounter issues depending on their environment. This section provides detailed troubleshooting strategies.

Missing libjpeg

If you see errors referencing jpeg functions:

```
sudo apt install libjpeg-dev
```

Missing GMP

If GMP headers or symbols are missing:

```
sudo apt install libgmp-dev
```

Undefined references

Often caused by incorrect link order. Ensure `-lgmp` and `-ljpeg` appear *after* source files.

Permission issues

If the build directory is read-only, move the source tree to a writable location.

Compiler incompatibilities

Older compilers may not support certain flags. Remove or adjust flags as needed.

Troubleshooting is part of the forensic mindset — understanding how tools behave under different conditions helps analysts build intuition about system behavior.

4.6 Running Opus

Once compiled, Opus can be executed directly:

```
./Opus
```

Upon execution, Opus performs several initialization steps:

1. **Terminal detection**
   Ensures ANSI support for menus and status lines.
2. **Thread pool initialization**
   Prepares worker threads for cryptanalysis tasks.
3. **Splash banner rendering**
   Displays the Opus logo and version information.
4. **Menu system initialization**
   Loads command categories and descriptions.
5. **Command prompt activation**
   Enters the interactive loop.

These steps ensure that Opus is fully prepared to handle tasks such as **recursive extraction**, **entropy analysis**, and **cryptanalysis workflows**.

4.7 Understanding the Startup Sequence

The startup sequence is designed to be both informative and reassuring. Analysts should always know that Opus is ready, stable, and functioning correctly before beginning analysis.

Splash Banner

The splash banner serves as a visual confirmation that Opus has initialized successfully. It also reinforces the identity of the toolkit and provides version information.

Thread Pool Activation

The thread pool is initialized early to ensure that long-running tasks can be dispatched immediately. This design supports responsiveness during operations such as **quadgram hillclimbing**.

Menu Display

The menu provides a structured overview of available commands, grouped by category. This reinforces discoverability and helps analysts navigate the toolkit.

Prompt Activation

The command prompt is the primary interface for interacting with Opus. It is designed to be simple, responsive, and consistent.

4.8 Environment Considerations

Opus behaves consistently across environments, but certain factors can influence performance or usability.

Terminal Size

A larger terminal improves readability, especially for diagrams and multi-line output.

File Permissions

Opus respects system permissions and will not override them. Analysts should ensure they have read access to files under investigation.

Directory Structure

Opus creates directories such as `carved/` and `extracted/` during recursive extraction. Analysts should ensure they have write access to the working directory.

Locale Settings

UTF-8 locales are recommended for proper string extraction and display.

4.9 Verifying Installation

After installation, analysts can verify that Opus is functioning correctly by running:

```
HELP
```

This command displays the full list of available commands and confirms that the menu system is operational.

Analysts may also test subsystems individually:

- **MD5 test**
- **ENTROPY test**
- **S test**
- **A test**
- **VIG test**

These tests help confirm that Opus is ready for real-world analysis.

==========

CHAPTER 5 — INTERFACE OVERVIEW

==========

Opus is a command-line forensic suite, but its interface is far more than a simple prompt. It is a carefully designed environment built to support clarity, discoverability, and analytical flow. This chapter provides a comprehensive overview of the Opus interface, explaining how each component contributes to the user experience and how analysts can leverage the interface to navigate complex workflows. While Opus does not use a graphical interface, its command-line environment is structured, expressive, and intentionally crafted to support deep forensic analysis.

5.1 The Philosophy of the Opus Interface

The Opus interface is built around several guiding principles that shape every aspect of its design:

Clarity

Every message printed by Opus is designed to be readable, unambiguous, and informative. Whether Opus is performing **recursive extraction**, **entropy analysis**, or **cryptanalysis workflows**, the interface ensures that analysts always understand what is happening.

Discoverability

The interface is structured so that analysts can explore Opus naturally. Commands are grouped into categories, menus are easy to navigate, and help messages are detailed and consistent. This design encourages analysts to experiment and learn without fear of breaking the system.

Minimalism

Opus avoids unnecessary visual clutter. The interface is clean, focused, and free of distractions. This minimalism supports deep analytical work, allowing analysts to focus on the data rather than the tool.

Responsiveness

The interface remains responsive even during long-running tasks. The **status line**, **thread pool**, and **background task queue** work together to ensure that Opus never feels frozen or unresponsive.

Consistency

Every command follows a consistent pattern of input, output, and error handling. This consistency reduces cognitive load and helps analysts build intuition about how Opus behaves.

5.2 The Splash Banner

When Opus starts, it displays a splash banner — a centered ASCII logo that serves as both a visual anchor and a confirmation that the toolkit has initialized successfully. The splash banner is more than decoration; it is a statement of identity, a reminder that Opus is a cohesive forensic environment with a clear purpose.

A typical splash banner might look like:

```
============================================================
                          OPUS v0.9
         Forensic, Steganographic & Cryptanalytic Toolkit
============================================================
```

The banner also reinforces versioning, which is essential for reproducibility and documentation. Analysts working on long-term investigations or CTF writeups can reference the version number to ensure consistency across environments.

5.3 The Main Menu

After the splash banner, Opus displays the main menu — a structured overview of available commands, grouped by category. The menu is designed to be both a reference and a navigation tool, helping analysts explore Opus's capabilities without memorizing every command.

A typical menu includes categories such as:

- File Operations
- Forensic Analysis
- Steganography
- Cryptanalysis
- Binary Tools
- Designation Tools
- RSA Tools
- Utility Commands

Each category contains a list of commands with brief descriptions. This structure supports **discoverability**, allowing analysts to quickly identify the tools they need.

The menu is intentionally simple, avoiding nested submenus or complex navigation. Analysts can return to the menu at any time by typing:

```
MENU
```

This reinforces the idea that Opus is a **guided environment**, not a collection of isolated utilities.

5.4 The Command Prompt

The command prompt is the primary interface for interacting with Opus. It is designed to be simple, responsive, and consistent. Analysts enter commands directly, and Opus responds with clear, structured output.

The prompt typically appears as:

```
Opus>
```

This prompt is intentionally minimal, allowing analysts to focus on the task at hand. It supports:

- single-letter commands
- multi-argument commands
- interactive prompts
- background task submission
- error messages
- status updates

The command parser is robust and forgiving, providing helpful feedback when commands are misused or arguments are missing. This design supports **analytical flow**, allowing analysts to experiment without fear of breaking the system.

5.5 Output Formatting

Opus uses a consistent output format across all commands. This consistency is essential for readability, especially during complex workflows such as **multi-layer extraction** or **cryptanalysis refinement**.

Output formatting includes:

Headers

Used to separate sections of output.

Indented Blocks

Used for nested information, such as carved files or entropy heatmaps.

Monospaced Blocks

Used for file contents, hex dumps, and diagrams.

Status Messages

Used to indicate progress, success, or failure.

Warnings

Used to highlight potential issues or unsafe operations.

This formatting ensures that analysts can quickly interpret results, even when working with large or complex datasets.

5.6 The Status Line

The status line is one of the most distinctive features of the Opus interface. It provides real-time updates on background tasks, particularly during long-running cryptanalysis operations.

A typical status line might display:

- number of active solver threads
- current restart count
- best score achieved
- progress text

For example:

```
[Threads: 4] [Restart: 128] [Best Score: -3421] [Status: Refining key...]
```

This line updates dynamically, giving analysts immediate feedback on the progress of tasks such as **quadgram hillclimbing** or **Vigenère refinement**.

The status line is designed to be unobtrusive, appearing at the bottom of the screen without interfering with other output. It reinforces the idea that Opus is a **living environment**, constantly working in the background to support analysis.

5.7 Background Task Queue

Opus includes a background task queue that manages long-running operations. This queue allows analysts to continue interacting with the interface while tasks run in parallel.

For example, an analyst might:

- start a Vigenère solver
- run entropy analysis on another file
- inspect strings
- open a new file

All while the solver continues running in the background.

This design supports **parallel analysis**, a key feature for forensic workflows where multiple hypotheses may need to be tested simultaneously.

5.8 Error Messages and Warnings

Opus provides clear, consistent error messages and warnings. These messages are designed to be informative without being overwhelming.

Examples include:

- missing arguments
- invalid file paths
- unsupported formats
- unsafe operations
- recursion guard triggers

Each message includes:

- a description of the issue
- the likely cause
- suggested corrective actions

This approach supports **analytical clarity**, helping analysts understand not only what went wrong, but why.

5.9 Interactive Prompts

Some commands require additional input from the analyst. In these cases, Opus uses interactive prompts that guide the user through the process.

For example:

```
Enter filename:
Enter key length:
Enter output path:
```

These prompts are designed to be clear and unambiguous, reducing the likelihood of errors.

5.10 The Opus Interface as an Analytical Environment

The Opus interface is more than a command prompt; it is an **analytical environment** designed to support deep, methodical investigation. Its structure encourages analysts to:

- explore data recursively
- test hypotheses iteratively
- interpret results critically
- maintain situational awareness
- work efficiently across multiple tasks

This environment is especially valuable during complex workflows such as **multi-layer extraction**, **JPEG steganalysis**, and **cryptanalysis refinement**.

==========

CHAPTER 6 — COMMAND REFERENCE

==========

This chapter provides a comprehensive, deeply detailed reference for every command available in Opus. Each command is documented with:

- syntax
- long-form explanation
- internal behavior
- rationale
- examples
- edge cases
- warnings
- notes
- related commands

This chapter is intentionally exhaustive. It is designed to be the definitive reference for analysts who want to understand not only how to use Opus, but why each command behaves the way it does.

==========

6.1 FILE OPERATIONS

==========

File operations form the backbone of Opus. These commands allow analysts to create, open, copy, hash, delete, and inspect files. While these operations may appear simple at first glance, Opus implements them with forensic rigor, ensuring safety, transparency, and reproducibility. This section explores each file operation in depth, providing detailed explanations and examples that illustrate their role in the broader Opus workflow.

6.1.1 CREATE — Create a New File

Syntax

```
CREATE <filename>
```

Description

The `CREATE` command creates a new, empty file in the current working directory. While this may seem like a trivial operation, Opus implements it with strict safety guarantees to prevent accidental overwrites or data loss. When an analyst issues the `CREATE` command, Opus performs a series of checks to ensure that the filename is valid, that the file does not already exist, and that the working directory is writable. Only after these checks pass does Opus create the file.

This command is particularly useful when preparing placeholder files, constructing test cases, or creating output targets for other commands. It also serves as a simple way to verify that Opus has write access to the current directory — a common issue in forensic environments where permissions may be restricted.

Internal Behavior

When `CREATE` is executed, Opus:

- **validates the filename**
- **checks for existing files**
- **opens the file in exclusive creation mode**
- **writes zero bytes**
- **closes the file safely**

These steps ensure that the file is created cleanly and without side effects.

Example

```
CREATE notes.txt
```

Edge Cases

- If the file already exists, Opus refuses to overwrite it.
- If the filename contains invalid characters, Opus prints an error.
- If the directory is read-only, Opus reports a permission error.

Warnings

Creating files in system directories may require elevated permissions. Analysts should ensure they are working in a safe, isolated environment.

6.1.2 COPY — Safe File Copy with MD5 Verification

Syntax

```
COPY <source> <destination>
```

Description

The `COPY` command performs a forensic-grade file copy operation. Unlike typical copy utilities, Opus implements a multi-stage workflow that ensures the integrity of the copied file. After copying the file in fixed-size chunks, Opus computes the MD5 hash of both the source and destination files and compares them. Only if the hashes match does Opus report success.

This approach ensures that analysts can trust the copied file, which is essential in forensic workflows where data integrity is paramount.

Internal Behavior

The `COPY` command performs the following steps:

- **opens the source file safely**
- **creates the destination file in exclusive mode**
- **copies data in fixed-size chunks**
- **computes MD5(source)**
- **computes MD5(destination)**
- **compares the hashes**
- **reports success or failure**

This workflow ensures that the destination file is an exact, byte-for-byte replica of the source.

Example

```
COPY secret.bin backup/secret_copy.bin
```

Edge Cases

- If the destination file exists, Opus refuses to overwrite it.
- If the source file is unreadable, Opus prints a detailed error.
- If the MD5 hashes do not match, Opus warns the analyst and deletes the incomplete copy.

Warnings

Copying large files may take time, especially on slow storage devices. Analysts should monitor the status line for progress updates.

6.1.3 MD5 — Compute MD5 Hash

Syntax

```
MD5 <filename>
```

Description

The `MD5` command computes the MD5 hash of a file and prints it in hexadecimal format. This command is essential for verifying file integrity, comparing files, and documenting forensic workflows. While MD5 is not suitable for cryptographic security, it remains widely used in digital forensics due to its speed and convenience.

Internal Behavior

Opus:

- **opens the file safely**
- **reads the file in chunks**
- **updates the MD5 context**
- **prints the final hash**

Example

```
MD5 payload.bin
```

Edge Cases

- If the file is empty, Opus prints the MD5 of an empty stream.
- If the file cannot be opened, Opus prints an error.

6.1.4 EXTRACT — Recursive Extraction Engine

Syntax

```
EXTRACT
```

Description

The `EXTRACT` command triggers the full recursive extraction pipeline — one of the most powerful features in Opus. This command analyzes the currently loaded file, detects compressed formats, decompresses them, normalizes suffixes, carves embedded files, analyzes each layer, and recurses into newly discovered files. The result is a complete, multi-layer forensic unwrapping of the input.

This command is essential for analyzing complex CTF challenges, malware samples, and multi-layer archives.

Internal Behavior

The extraction engine performs:

- **format detection**
- **decompression**
- **suffix normalization**
- **file carving**
- **string intelligence**
- **JPEG analysis**
- **recursion into carved files**

This pipeline continues until no further formats are detected.

Example

```
EXTRACT
```

Edge Cases

- Corrupted archives may partially extract.
- Infinite recursion is prevented by recursion guards.
- Carved files are placed in the `carved/` directory.

6.1.5 DEL — Secure Delete

Description

The `DEL` command securely deletes the currently loaded file using multi-pass overwrite patterns. This ensures that sensitive data cannot be recovered using simple forensic tools.

Internal Behavior

Opus:

- **overwrites the file with patterns**
- **flushes buffers**
- **deletes the file**

Warnings

Secure deletion is irreversible.

6.1.6 OPEN — Open File

Prompts the analyst for a filename and loads it as the current file.

Internal Behavior

- **validates path**
- **opens file safely**
- **updates internal state**

6.1.7 READ — Read File

Displays the contents of the current file.

Internal Behavior

- **reads file in chunks**
- **prints safely**

6.1.8 CLR — Clear Screen

Clears the terminal.

6.1.9 EXIT / Z — Exit Opus

Shuts down the thread pool and exits cleanly.

==========

6.2 FORENSIC ANALYSIS COMMANDS

==========

Forensic analysis is one of the core pillars of Opus. These commands allow analysts to examine files at a structural, statistical, and semantic level, revealing patterns, anomalies, and embedded content that may not be visible through simple inspection. This section provides an exhaustive reference for each forensic analysis command, including long-form explanations, internal behavior, examples, edge cases, warnings, and rationale.

Opus's forensic tools are designed to support a wide range of investigative workflows, from **entropy interpretation** to **string extraction** to **magic byte detection** and beyond. Each tool is built with forensic rigor, ensuring that results are reliable, reproducible, and meaningful.

==========

6.2.1 ENTROPY — Global & Block-Level Entropy Analysis

==========

Syntax

```
ENTROPY
```

Description

The `ENTROPY` command computes the Shannon entropy of the currently loaded file. Entropy is a statistical measure of randomness, and it is one of the most powerful tools in digital forensics. High entropy often indicates compressed or encrypted data, while low entropy may indicate structured or human-readable content. Opus computes both **global entropy** and **block-level entropy**, providing a detailed view of the file's internal structure.

Entropy analysis is essential for identifying hidden content, detecting steganographic modifications, and guiding further investigation. For example, a JPEG file with unusually high entropy in its DCT coefficient blocks may indicate the presence of hidden data. Similarly, a binary with low-entropy padding may reveal embedded strings or metadata.

Internal Behavior

When `ENTROPY` is executed, Opus performs the following steps:

- **reads the file in fixed-size blocks**
- **computes byte frequency distributions**
- **calculates Shannon entropy for each block**
- **computes global entropy across the entire file**
- **prints a block-level heatmap**
- **prints summary statistics**

This multi-stage workflow provides both macro-level and micro-level insight into the file's structure.

Example Output

```
Global Entropy: 7.92 bits/byte
Block Size: 4096 bytes

Block 0: 7.88
Block 1: 7.91
Block 2: 7.93
Block 3: 7.12  <-- anomaly
Block 4: 7.94
```

Interpretation

- **7.5–8.0 bits/byte** → likely compressed or encrypted
- **5.0–7.0 bits/byte** → mixed content
- **<5.0 bits/byte** → structured or human-readable data

Edge Cases

- Very small files may produce misleading entropy values.
- Files with uniform data (e.g., all zeros) have entropy near 0.
- Files with random noise have entropy near 8.

Warnings

Entropy alone cannot distinguish between encryption and compression. Analysts should combine entropy analysis with **string intelligence** and **magic byte detection**.

==========

6.2.2 STRING — String Intelligence Engine

==========

Syntax

`STRING`

Description

The `S` command invokes the **String Intelligence Engine**, one of the most informative tools in Opus. Unlike traditional `strings` utilities, Opus performs a multi-stage analysis that detects:

- **ASCII strings**
- **UTF-8 sequences**
- **base64 candidates**
- **URLs**
- **email addresses**
- **offsets for each string**

This enhanced approach provides analysts with a richer understanding of the file's semantic content. For example, base64 strings may indicate encoded payloads, while URLs may reveal command-and-control infrastructure.

Internal Behavior

The String Intelligence Engine performs:

- **byte-by-byte scanning**
- **ASCII detection**
- **UTF-8 validation**
- **base64 heuristics**
- **URL/email extraction**
- **offset annotation**

This multi-layered approach ensures that Opus captures meaningful content even in noisy or partially corrupted files.

Example Output

```
[0x0012A0] "GET /index.php HTTP/1.1"
[0x004F20] "admin@example.com"
[0x00A120] "aGVsbG8gd29ybGQ=" (base64)
```

Edge Cases

- Binary files may contain false positives.
- UTF-8 detection may skip invalid sequences.
- Base64 heuristics may detect short fragments.

Warnings

Analysts should verify base64 strings manually before decoding.

==========

6.2.3 MAGIC — Magic Byte Detector

==========

Syntax

```
MAGIC
```

Description

The `MAGIC` command scans the file for known magic bytes — signature patterns that identify file formats. This tool is essential for detecting embedded files, corrupted structures, and disguised payloads. Magic byte detection is often the first step in identifying hidden content.

Internal Behavior

Opus scans the file for:

- **JPEG signatures**
- **PNG signatures**
- **ZIP headers**
- **ELF headers**
- **GZIP headers**
- **custom signatures**

When a signature is found, Opus prints the offset and the detected format.

Example Output

```
[0x000000] Detected PNG header
[0x001200] Detected ZIP local header
```

Edge Cases

- Corrupted signatures may be partially detected.
- False positives may occur in random data.

==========

6.2.4 A — ASCII Dump

==========

Syntax

```
A
```

Description

The `A` command prints an ASCII-only view of the file, replacing non-printable characters with dots. This view is useful for identifying readable content embedded within binary data.

Internal Behavior

- **reads file in chunks**
- **filters printable characters**
- **prints ASCII view**

Example Output

```
Hello....world....PNG....IHDR....
```

==========

6.2.5 HEX — Hex Dump

==========

Syntax

`HEX`

Description

The `HEX` command prints a traditional hex dump of the file, including offsets, hex bytes, and ASCII equivalents. This tool is essential for low-level inspection.

Internal Behavior

- **reads file in 16-byte blocks**
- **prints offset**
- **prints hex bytes**
- **prints ASCII column**

==========

6.2.6 FREQ — Byte Frequency Analysis

==========

Syntax

`FREQ`

Description

The `FREQ` command computes the frequency of each byte value (0–255) in the file. This analysis is useful for identifying patterns, anomalies, and potential steganographic modifications.

Internal Behavior

- **counts occurrences of each byte**

- **prints histogram**
- **computes summary statistics**

==========

6.3 STEGANOGRAPHY COMMANDS

==========

Steganography analysis is one of the most specialized and technically demanding areas of digital forensics. Unlike cryptography, which hides the *meaning* of data, steganography hides the *existence* of data. This makes detection inherently more subtle, more statistical, and more dependent on understanding the structure of the underlying media. Opus includes a full suite of JPEG-focused steganalysis tools designed to detect anomalies in bit-planes, coefficient distributions, and encoding structures. These tools are not intended to “magically extract” hidden payloads; instead, they provide analysts with the statistical and structural evidence needed to determine whether a file has been manipulated.

This section provides an exhaustive reference for each steganography command in Opus, including long-form explanations, internal behavior, examples, edge cases, warnings, and forensic rationale.

==========

6.3.1 J — JPEG Steganalysis Overview

==========

Syntax

```
J
```

Description

The `J` command serves as the entry point to the JPEG Stego Engine. When invoked, Opus performs a multi-stage analysis of the currently loaded JPEG file, examining everything from bit-plane irregularities to DCT coefficient anomalies. This command is designed to give analysts a high-level

overview of the file's steganographic profile, summarizing the results of several deeper analyses that can be invoked individually.

The JPEG Stego Engine is one of the most complex subsystems in Opus. It integrates statistical tests, structural analysis, and heuristic scoring to provide a comprehensive assessment of whether a JPEG file may contain hidden data. This command is particularly useful when triaging large numbers of images or when beginning a detailed investigation.

Internal Behavior

When `J` is executed, Opus performs:

- **LSB bit-plane sampling**
- **chi-square deviation testing**
- **RS (Regular/Singular) analysis**
- **DCT coefficient profiling**
- **Huffman table fingerprinting**

Each of these tests contributes to a composite "stego suspicion score," which Opus prints at the end of the analysis.

Example Output

```
JPEG Stego Summary:
 - LSB Analysis: Mild anomalies detected
 - Chi-Square Test: Deviations in block group 12
 - RS Analysis: Regular/Singular imbalance detected
 - DCT Profiling: High-frequency coefficient anomalies
 - Huffman Fingerprint: Non-standard table detected

Overall Suspicion: HIGH
```

Edge Cases

- Highly compressed JPEGs may produce false positives.
- Images with heavy post-processing may appear anomalous.
- Some stego tools modify only specific channels or blocks.

Warnings

The `J` command does **not** extract hidden data. It only detects anomalies.

==========

6.3.2 JLSB — LSB Bit-Plane Analysis

==========

Syntax

```
JLSB
```

Description

The `JLSB` command performs a detailed analysis of the least significant bits (LSBs) in the JPEG's decompressed pixel data. While JPEG is a lossy format, many steganographic tools operate by modifying the LSBs of the decompressed image, relying on the fact that these changes are visually imperceptible. Opus examines these bit-planes for statistical irregularities that may indicate hidden data.

Internal Behavior

Opus performs:

- **bit-plane extraction**
- **histogram comparison**
- **randomness testing**
- **block-level anomaly detection**

The engine compares the observed LSB distribution against expected patterns derived from natural images.

Example Output

```
LSB Analysis:
 - Bit-plane 0: Uniform distribution (suspicious)
 - Bit-plane 1: Natural variation
 - Block anomalies: 3 blocks exceed threshold
```

Edge Cases

- Synthetic images may naturally have uniform LSBs.
- Heavy compression may distort LSB patterns.

Warnings

LSB anomalies alone do not prove steganography; they only indicate suspicion.

==========

6.3.3 JCHI — Chi-Square Stego Test

==========

Syntax

```
JCHI
```

Description

The `JCHI` command performs chi-square testing on the JPEG's decompressed pixel data. This statistical test compares the observed distribution of pixel values to the expected distribution for natural images. Significant deviations may indicate that the image has been modified to embed hidden data.

Internal Behavior

Opus performs:

- **pixel histogram generation**
- **expected distribution modeling**
- **chi-square scoring**
- **block-level deviation mapping**

Example Output

```
Chi-Square Test:
 - Global deviation: HIGH
 - Block group 12: Significant anomaly
 - Block group 27: Moderate anomaly
```

Edge Cases

- Over-edited images may produce false positives.
- Low-quality JPEGs may distort expected distributions.

==========

6.3.4 JRS — Regular/Singular (RS) Analysis

==========

Syntax

```
JRS
```

Description

The `JRS` command performs RS analysis, a powerful statistical technique for detecting LSB steganography. RS analysis divides the image into groups of pixels and measures how flipping LSBs affects the smoothness of each group. Natural images exhibit predictable patterns; stego images do not.

Internal Behavior

Opus performs:

- **group partitioning**
- **smoothness measurement**
- **LSB flipping simulation**
- **regular/singular classification**

Example Output

```
RS Analysis:
 - Regular groups: 12412
 - Singular groups: 11890
 - Imbalance: HIGH
```

Edge Cases

- Images with sharp edges may skew results.
- Very small images may not produce enough groups.

==========

6.3.5 JDCT — DCT Coefficient Profiling

==========

Syntax

```
JDCT
```

Description

The `JDCT` command analyzes the JPEG's DCT (Discrete Cosine Transform) coefficients. Many steganographic tools operate directly on DCT coefficients, modifying high-frequency components to embed data. Opus profiles these coefficients to detect anomalies.

Internal Behavior

Opus performs:

- **coefficient extraction**
- **frequency band analysis**
- **histogram comparison**
- **high-frequency anomaly detection**

Example Output

```
DCT Profiling:
 - High-frequency anomalies detected
 - Coefficient band (5,7) shows irregular distribution
```

Edge Cases

- Low-quality JPEGs may naturally distort high-frequency coefficients.

==========

6.3.6 JHUF — Huffman Table Fingerprinting

==========

Syntax

`JHUF`

Description

The `JHUF` command analyzes the JPEG's Huffman tables. Many steganographic tools modify these tables to optimize embedding or disguise payloads. Opus compares the observed tables against known standards.

Internal Behavior

Opus performs:

- **Huffman table extraction**
- **standard table comparison**
- **entropy coding analysis**
- **anomaly scoring**

Example Output

```
Huffman Fingerprint:
 - Table 0: Non-standard
 - Table 1: Standard
 - Suspicion: MODERATE
```

Edge Cases

- Some cameras use non-standard tables legitimately.

==========

6.4 CRYPTANALYSIS COMMANDS

==========

Cryptanalysis is one of the most powerful and sophisticated capabilities in Opus. Unlike forensic analysis, which focuses on structure and content, cryptanalysis focuses on **meaning**, attempting to recover plaintext from ciphertext through statistical, structural, or mathematical methods. Opus includes tools for classical ciphers, modern scoring systems, and large-integer factorization. These tools are designed to support CTF challenges, academic exercises, and real-world investigative workflows.

This section provides an exhaustive reference for each cryptanalysis command, including long-form explanations, internal behavior, examples, edge cases, warnings, and rationale.

==========

6.4.1 VIG — Vigenère Cipher Solver

==========

Syntax

```
VIG
```

Description

The `VIG` command invokes the Vigenère Solver, a multi-stage cryptanalysis engine designed to break polyalphabetic substitution ciphers. Unlike simple brute-force tools, Opus uses a combination of **Index of Coincidence**, **Kasiski examination**, **harmonic scoring**, and **hillclimb refinement** to recover both the key length and the key itself.

The solver is designed to handle ciphertexts of varying lengths, noise levels, and alphabet distributions. It is particularly effective in CTF challenges where ciphertexts are long enough to support statistical analysis.

Internal Behavior

When `VIG` is executed, Opus performs:

- **key-length estimation** using IC and Kasiski
- **initial key guess generation**
- **hillclimb refinement** using quadgram scoring
- **mutation cycles** to escape local maxima
- **multi-threaded restarts** for robustness
- **best-score tracking**

This workflow ensures that Opus converges on the most statistically likely plaintext.

Example Output

```
Estimated Key Length: 7
Initial Key Guess: QWERTYU
Refining...
Best Key: CRYPTOG
Best Score: -4123
Plaintext:
THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG
```

Edge Cases

- Very short ciphertexts may produce unreliable key-length estimates.
- Non-English plaintexts may require custom quadgram tables.
- Mixed-alphabet ciphertexts may confuse the solver.

Warnings

The Vigenère solver assumes English plaintext unless configured otherwise.

==========

6.4.2 VIGLEN — Key-Length Estimation

==========

Syntax

```
VIGLEN
```

Description

The `VIGLEN` command performs key-length estimation without attempting to solve the cipher. This is useful when analysts want to manually inspect the ciphertext or prepare for a more targeted attack.

Internal Behavior

Opus performs:

- **Index of Coincidence analysis**
- **Kasiski examination**
- **harmonic scoring**
- **length ranking**

Example Output

```
Likely Key Lengths:
 7 (highest likelihood)
 14
 21
```

==========

6.4.3 SUB — Substitution Cipher Solver

==========

Syntax

`SUB`

Description

The `SUB` command invokes the monoalphabetic substitution solver, a powerful hillclimbing engine that uses **quadgram scoring**, **mutation cycles**, and **multi-threaded restarts** to recover plaintext from substitution ciphers.

This solver is designed to handle ciphertexts with:

- letter frequency distortion
- partial corruption
- non-standard spacing
- mixed punctuation

It is one of the most computationally intensive tools in Opus.

Internal Behavior

Opus performs:

- **frequency-based initial key guess**
- **hillclimb refinement**
- **swap, rotate, and perturb mutations**
- **quadgram scoring**
- **accept/reject cycles**
- **multi-threaded restarts**

Example Output

```
Initial Key Guess: QWERTYUIOPASDFGHJKLZXCVBNM
Refining...
Best Key: ZEBRASCDFGHIJKLMNOPQTUVWXY
Plaintext:
ATTACK AT DAWN STOP RENDEZVOUS AT POINT BRAVO
```

Edge Cases

- Very short ciphertexts may not converge.
- Ciphertexts with non-English plaintext may require custom scoring tables.

==========

6.4.4 FREQ — Frequency Analysis

==========

Syntax

`FREQ`

Description

The `FREQ` command performs classical frequency analysis on the ciphertext. This tool is essential for understanding the structure of substitution ciphers and guiding manual analysis.

Internal Behavior

Opus:

- **counts letter frequencies**
- **prints histogram**

- **computes chi-square deviation**

Example Output

```
E: 12.3%
T: 9.8%
A: 8.1%
```

==========

6.4.5 RSA — RSA Factorization Tool

==========

Syntax

```
RSA
```

Description

The `RSA` command invokes the RSA analysis engine, which attempts to factor the modulus `N` using a combination of:

- **Fermat search**
- **Pollard's Rho**
- **trial division for small primes**
- **GMP-accelerated arithmetic**

This tool is designed for CTF-style RSA challenges where `N` is intentionally weak.

Internal Behavior

Opus performs:

- **parity check**
- **difference-of-squares search**
- **random walk factorization**
- **cycle detection**
- **p/q recovery**
- **private exponent computation**

Example Output

```
N = 9456021049
```

```
Factoring...
p = 97213
q = 97307
d = 1234567
```

Edge Cases

- Large moduli may take extremely long to factor.
- Strong RSA keys cannot be broken.

==========

6.4.6 DESIG — Designation Block Analyzer

==========

Syntax

```
DESIG
```

Description

The `DESIG` command analyzes Opus designation blocks — structured metadata blocks used in certain workflows. This tool extracts fields, validates checksums, and prints detailed summaries.

Internal Behavior

Opus performs:

- **header parsing**
- **field extraction**
- **checksum validation**
- **structure verification**

==========

6.4.7 QUAD — Quadgram Scoring Tool

==========

Syntax

```
QUAD
```

Description

The `QUAD` command computes the quadgram score of a given text. This is primarily used for debugging or manually evaluating plaintext candidates.

Internal Behavior

Opus:

- **loads quadgram table**
- **computes log-likelihood**
- **prints score**

==========

6.5 BINARY TOOLS

==========

Binary analysis is a critical component of modern forensics and CTF workflows. While Opus is not a full disassembler or reverse-engineering framework, it includes several specialized tools designed to extract structural information from binaries, analyze execution-related metadata, and assist analysts in understanding how a binary behaves at runtime. These tools are intentionally lightweight, focusing on clarity and forensic relevance rather than deep symbolic analysis.

This section provides an exhaustive reference for each binary tool in Opus, including long-form explanations, internal behavior, examples, edge cases, warnings, and rationale.

==========

6.5.1 ELF — ELF Header & Section Inspector

==========

Syntax

`ELF`

Description

The `ELF` command inspects the structure of an ELF (Executable and Linkable Format) binary. ELF is the standard binary format on Unix-like systems, and understanding its structure is essential for reverse engineering, malware analysis, and exploit development. Opus provides a detailed breakdown of the ELF header, program headers, section headers, and symbol tables, giving analysts a clear view of the binary's internal layout.

This command is particularly useful when triaging unknown binaries, identifying architecture and ABI details, or preparing for deeper analysis in external tools.

Internal Behavior

When `ELF` is executed, Opus performs:

- **ELF magic verification**
- **header parsing**
- **program header enumeration**
- **section table parsing**
- **symbol table extraction**
- **string table resolution**

Opus prints each component in a structured, readable format, ensuring that analysts can interpret the results without needing to consult external documentation.

Example Output

```
ELF Header:
 - Class: ELF64
 - Endianness: Little
 - Type: DYN (Shared Object)
 - Machine: x86_64
 - Entry Point: 0x0000000000001130

Sections:
 [0] .text        0x0000000000002000  0x1A34
 [1] .data        0x0000000000004000  0x0040
```

```
[2] .rodata      0x0000000000005000  0x0120
[3] .symtab      0x0000000000006000  0x0800
```

Edge Cases

- Stripped binaries may lack symbol tables.
- Packed binaries may have misleading section sizes.
- Corrupted ELF headers may cause partial parsing.

Warnings

The `ELF` command does **not** disassemble code. It only inspects structure.

==========

6.5.2 PIECALC — PIE Base Estimation

==========

Syntax

`PIECALC`

Description

The `PIECALC` command estimates the PIE (Position-Independent Executable) base address of a binary using a known return address or leak. PIE binaries are loaded at randomized addresses due to ASLR (Address Space Layout Randomization), making exploitation more challenging. However, if an analyst obtains a leaked return address, Opus can estimate the PIE base by subtracting known offsets.

This tool is essential for exploit development, CTF pwn challenges, and understanding runtime memory layout.

Internal Behavior

Opus performs:

- **return address parsing**
- **offset matching**
- **heuristic scoring**
- **confidence estimation**
- **base address calculation**

The engine compares the leaked address against known offsets in the ELF file to determine the most likely PIE base.

Example Output

```
Return Address: 0x7ffff7dd18b3
Matched Offset: 0x00000000000108b3
Estimated PIE Base: 0x7ffff7dc1000
Confidence: HIGH
```

Edge Cases

- Incorrect leaks may produce misleading results.
- Stripped binaries reduce confidence.
- PIECALC cannot operate on non-PIE binaries.

Warnings

PIE base estimation is probabilistic. Analysts should verify results manually.

==========

6.5.3 RETTIME — Return-Address Timing Estimator

==========

Syntax

```
RETTIME
```

Description

The `RETTIME` command estimates the timing characteristics of return-address execution paths. This tool is primarily used in advanced CTF challenges where timing side-channels reveal information about control flow or memory layout. While not commonly used in everyday forensics, it is included in Opus for completeness and for analysts who work with microarchitectural puzzles.

Internal Behavior

Opus performs:

- **timed execution loops**

- **branch prediction sampling**
- **cache hit/miss profiling**
- **statistical averaging**

The output provides insight into whether a return address is likely valid, misaligned, or pointing into unmapped memory.

Example Output

```
Timing Profile:
 - Average cycles: 142
 - Variance: LOW
 - Interpretation: Likely valid return address
```

Edge Cases

- Virtualized environments may distort timing.
- CPU frequency scaling may affect results.

Warnings

Timing analysis is inherently noisy. Results should be interpreted cautiously.

==========

6.5.4 SYM — Symbol Table Browser

==========

Syntax

`SYM`

Description

The `SYM` command prints the symbol table of the currently loaded ELF file. This includes function names, global variables, and other symbols. Symbol tables are invaluable for reverse engineering, debugging, and understanding binary structure.

Internal Behavior

Opus performs:

- **symbol table extraction**

- **string table resolution**
- **symbol classification**
- **address printing**

Example Output

```
Symbols:
 0x0000000000001130  main
 0x0000000000001200  helper_function
 0x0000000000003000  global_counter
```

Edge Cases

- Stripped binaries may have no symbols.
- Obfuscated binaries may rename symbols.

==========

6.5.5 SECT — Section Table Browser

==========

Syntax

```
SECT
```

Description

The `SECT` command prints the section table of the ELF file, including names, offsets, sizes, and flags. This information is essential for understanding how the binary is organized.

Internal Behavior

Opus performs:

- **section header parsing**
- **flag decoding**
- **offset/size printing**

Example Output

```
[0] .text   0x2000  0x1A34  AX
[1] .data   0x4000  0x0040  WA
[2] .bss    0x5000  0x0100  WA
```

==========

6.6 DESIGNATION TOOLS

==========

Designation tools form a specialized subsystem within Opus, designed to parse, validate, and interpret **designation blocks** — structured metadata containers used in certain workflows to store auxiliary information, embedded parameters, or internal Opus-specific annotations. While these blocks are not commonly encountered in everyday forensic work, they play a crucial role in advanced Opus pipelines, especially when dealing with recursive extraction, multi-stage cryptanalysis, or Opus-generated artifacts.

This section provides a comprehensive reference for the designation tools, including long-form explanations, internal behavior, examples, edge cases, warnings, and rationale. Even though designation blocks are relatively rare, their structure and purpose are documented here with the same exhaustive detail as every other subsystem in Opus.

==========

6.6.1 DESIG — Designation Block Analyzer

==========

Syntax

```
DESIG
```

Description

The `DESIG` command analyzes the currently loaded file to determine whether it contains an Opus designation block. A designation block is a structured metadata segment that begins with a fixed header, followed by a series of typed fields, and ends with a checksum. These blocks are used internally by Opus to store information such as:

- **extraction lineage**
- **solver metadata**

- **analysis parameters**
- **embedded hints or annotations**

Designation blocks are intentionally simple, predictable, and self-describing. They are designed to be easy to parse, easy to validate, and easy to embed within larger files without interfering with surrounding data.

The `DESIG` command provides a full breakdown of the block, printing each field, its type, its length, and its interpreted value. This makes it possible for analysts to understand how Opus-generated artifacts were created, how they relate to one another, and how they should be interpreted in multi-stage workflows.

Internal Behavior

When `DESIG` is executed, Opus performs:

- **header verification**
- **field enumeration**
- **type decoding**
- **length validation**
- **checksum verification**
- **structure integrity checks**

If any part of the block is malformed, Opus prints a detailed error message explaining the nature of the corruption.

Example Output

```
Designation Block Detected
Header: OPUS-DESIG v1
Fields:
  [0] TYPE: SOURCE_FILE
      LENGTH: 12
      VALUE: payload.bin

  [1] TYPE: EXTRACTION_DEPTH
      LENGTH: 1
      VALUE: 3

  [2] TYPE: NOTES
      LENGTH: 42
      VALUE: "Recovered during recursive extraction stage 2"

Checksum: VALID
```

Edge Cases

- If the block is truncated, Opus prints a structural error.
- If the checksum fails, Opus marks the block as suspicious.
- If unknown field types are encountered, Opus prints them in raw form.
- If multiple designation blocks exist, Opus parses the first and warns about additional blocks.

Warnings

Designation blocks are **not** cryptographically secure. They are integrity-checked, not tamper-proof.

==========

6.6.2 DESIGRAW — Raw Designation Block Dump

==========

Syntax

```
DESIGRAW
```

Description

The `DESIGRAW` command prints the raw bytes of the designation block without attempting to interpret or validate them. This is useful when:

- **performing manual verification**
- **debugging malformed blocks**
- **extracting embedded payloads**
- **comparing block versions**

Unlike `DESIG`, which focuses on structure and interpretation, `DESIGRAW` focuses on raw visibility. It prints the block in a hex-dump format, allowing analysts to inspect the underlying data directly.

Internal Behavior

Opus performs:

- **header scan**
- **block length extraction**
- **raw byte dump**

No validation is performed. No interpretation is attempted. The output is purely structural.

Example Output

```
0000: 4F 50 55 53 2D 44 45 53 49 47 01 00 00 00 2A ...
0010: 01 0C 70 61 79 6C 6F 61 64 2E 62 69 6E 02 01 ...
```

Edge Cases

- If the block is malformed, the dump may include trailing garbage.
- If the block is extremely large, Opus paginates the output.
- If no block is found, Opus prints a simple “No designation block detected.”

Warnings

Raw dumps may contain binary data that is not safe to display in certain terminals.

==========

6.6.3 DESIGCHK — Designation Integrity Checker

==========

Syntax

```
DESIGCHK
```

Description

The `DESIGCHK` command performs a full integrity check of the designation block, verifying:

- **header correctness**
- **field alignment**
- **length consistency**
- **checksum validity**
- **version compatibility**

This command is particularly useful when analyzing Opus-generated artifacts that have passed through multiple transformations, such as compression, carving, or recursive extraction. It ensures that the block remains structurally sound and that no corruption has occurred.

Internal Behavior

Opus performs:

- **header parse**
- **field iteration**
- **checksum recomputation**
- **error reporting**

If any inconsistency is detected, Opus prints a detailed diagnostic report.

Example Output

```
Designation Block Integrity Check:
 - Header: VALID
 - Field Structure: VALID
 - Lengths: VALID
 - Checksum: VALID
 - Version: COMPATIBLE

Overall Status: PASS
```

Edge Cases

- Blocks with unknown field types may still pass integrity checks.
- Blocks with correct structure but incorrect semantics are marked as “structurally valid.”

Warnings

Integrity checks do **not** guarantee authenticity — only structural correctness.

==========

6.7 RSA TOOLS

==========

RSA analysis occupies a unique place in Opus. Unlike classical ciphers, which rely on linguistic structure, RSA is grounded in number theory, modular arithmetic, and the hardness of factoring large integers. In real-world cryptography, RSA is secure only when implemented with strong parameters — large primes, proper padding, and correct key generation. But in CTFs, academic exercises, and intentionally weakened challenges, RSA moduli are often constructed with exploitable properties.

Opus includes a suite of RSA tools designed to inspect, analyze, and — when possible — break RSA keys. These tools are not intended to replace full cryptographic libraries or symbolic algebra systems; instead, they provide analysts with a focused, forensic-oriented environment for understanding RSA structures, identifying weaknesses, and recovering private keys when feasible.

This section documents each RSA tool in exhaustive detail, including long-form explanations, internal behavior, examples, edge cases, warnings, and rationale.

==========

6.7.1 RSA — RSA Factorization Engine

==========

Syntax

```
RSA
```

Description

The `RSA` command is the primary entry point into the RSA analysis subsystem. When invoked, Opus attempts to factor the modulus `N` of the currently loaded RSA key or ciphertext container. This engine is designed for **CTF-style RSA challenges**, where the modulus is intentionally weak, misconstructed, or derived from primes with exploitable relationships.

Opus does **not** attempt to break real-world RSA keys. Instead, it focuses on:

- **small or medium-sized moduli**
- **close primes**
- **imbalanced primes**
- **Fermat-friendly structures**
- **moduli with detectable algebraic weaknesses**

The RSA engine uses a combination of classical and modern factorization techniques, all powered by GMP for high-precision arithmetic.

Internal Behavior

When `RSA` is executed, Opus performs:

- **modulus parsing**
- **bit-length analysis**
- **parity checks**
- **trial division for small primes**
- **Fermat difference-of-squares search**
- **Pollard’s Rho random walk**
- **cycle detection**
- **p/q recovery**
- **Euler totient computation**
- **private exponent derivation**

This multi-stage workflow ensures that Opus can handle a wide range of RSA challenges, from trivial to moderately complex.

Example Output

```
Modulus (N): 9456021049
Bit Length: 34 bits
Factoring...
p = 97213
q = 97307
phi(N) = 9455821520
Private Exponent (d) = 1234567
```

Edge Cases

- If `N` is prime, Opus reports that factorization is impossible.
- If `N` is too large, Opus warns that factorization may take an impractical amount of time.
- If `N` is malformed, Opus prints a structural error.
- If `N` is a perfect square, Opus detects and reports it immediately.

Warnings

The RSA engine is **not** a general-purpose factorization tool. It is optimized for CTF-style weaknesses, not real-world cryptographic keys.

==========

6.7.2 RSAPUB — RSA Public Key Inspector

==========

Syntax

```
RSAPUB
```

Description

The `RSAPUB` command inspects the structure of an RSA public key, printing detailed information about:

- **modulus (N)**
- **public exponent (e)**
- **bit length**
- **parity**
- **structural anomalies**

This tool is essential for triaging RSA challenges, identifying potential weaknesses, and preparing for factorization attempts. It also helps analysts understand whether a given key is likely to be breakable using Opus's factorization engine.

Internal Behavior

Opus performs:

- **PEM/DER parsing**
- **ASN.1 structure decoding**
- **modulus extraction**
- **exponent extraction**
- **bit-length computation**
- **sanity checks**

Example Output

```
RSA Public Key:
 - Modulus (N): 9456021049
 - Bit Length: 34 bits
 - Exponent (e): 65537
 - Structure: VALID
 - Notes: Modulus is unusually small
```

Edge Cases

- Keys with non-standard exponents may trigger warnings.
- Corrupted PEM files may partially parse.
- DER-encoded keys may contain padding anomalies.

Warnings

RSAPUB does **not** attempt factorization — it only inspects structure.

==========

6.7.3 RSAPRIV — RSA Private Key Inspector

==========

Syntax

```
RSAPRIV
```

Description

The `RSAPRIV` command inspects RSA private keys, printing:

- **modulus (N)**
- **public exponent (e)**
- **private exponent (d)**
- **prime factors (p, q)**
- **CRT parameters (dp, dq, qinv)**

This tool is useful for verifying the correctness of recovered keys, validating CTF challenge solutions, or inspecting Opus-generated RSA artifacts.

Internal Behavior

Opus performs:

- **PEM/DER parsing**
- **ASN.1 decoding**
- **parameter extraction**
- **consistency checks**
- **CRT validation**

Example Output

```
RSA Private Key:
 - N: 9456021049
 - e: 65537
 - d: 1234567
 - p: 97213
 - q: 97307
 - dp: 12345
 - dq: 67890
 - qinv: 54321
Structure: VALID
```

Edge Cases

- Corrupted private keys may fail ASN.1 decoding.
- Keys missing CRT parameters may still be valid but slower to use.
- Keys with mismatched parameters are flagged as invalid.

Warnings

Private keys should be handled with care. Opus does not store or transmit key material.

==========

6.7.4 RSAMOD — Modulus Structure Analyzer

==========

Syntax

`RSAMOD`

Description

The `RSAMOD` command analyzes the structure of the RSA modulus `N`, looking for:

- **perfect squares**
- **imbalanced primes**
- **close primes**
- **smoothness**
- **Fermat-friendly structure**
- **small prime factors**

This tool helps analysts determine whether `N` is likely to be breakable using Opus's factorization engine.

Internal Behavior

Opus performs:

- **square test**
- **difference-of-squares heuristic**
- **trial division**
- **smoothness estimation**
- **bit-length analysis**

Example Output

```
Modulus Analysis:
 - Bit Length: 34 bits
 - Perfect Square: NO
 - Close Primes: YES (difference = 94)
 - Smoothness: HIGH
 - Likely Breakable: YES
```

Edge Cases

- Very large moduli may produce inconclusive smoothness estimates.

- Moduli with exotic structures may confuse heuristics.

==========

6.7.5 RSACRT — CRT Parameter Validator

Syntax

```
RSACRT
```

Description

The RSACRT command validates the Chinese Remainder Theorem parameters of an RSA private key. These parameters dramatically speed up RSA decryption and signing, but only if they are correct.

Internal Behavior

Opus performs:

- **dp/dq consistency checks**
- **qinv validation**
- **modular arithmetic verification**

Example Output

```
CRT Validation:
 - dp: VALID
 - dq: VALID
 - qinv: VALID
Overall Status: PASS
```

==========

6.8 UTILITY COMMANDS

==========

Utility commands form the connective tissue of Opus — the small but essential operations that support navigation, configuration, workflow management, and user experience. While these commands do not perform forensic or cryptanalytic analysis directly, they are crucial for maintaining analytical flow,

ensuring clarity, and providing a stable environment for deeper operations. In many ways, these commands are the “quality-of-life” features that make Opus feel like a cohesive forensic suite rather than a loose collection of tools.

This section documents each utility command in exhaustive detail, including long-form explanations, internal behavior, examples, edge cases, warnings, and rationale.

==========

6.8.1 HELP — Command Reference Overview

==========

Syntax

```
HELP
```

Description

The `HELP` command prints a complete list of available commands, grouped by category. This is the primary discovery mechanism for new users and a quick reference for experienced analysts. Unlike the menu displayed at startup, `HELP` can be invoked at any time, making it a reliable anchor point during complex workflows.

The output is intentionally verbose, providing both command names and short descriptions. This reinforces Opus’s philosophy of **discoverability** and ensures that analysts never feel lost, even when navigating the more advanced subsystems.

Internal Behavior

When `HELP` is executed, Opus performs:

- **category enumeration**
- **command listing**
- **description formatting**
- **output pagination (if needed)**

Example Output

```
File Operations:
  CREATE   Create a new file
  COPY     Copy file with MD5 verification
  MD5      Compute MD5 hash
  ...
```

```
Forensic Analysis:
  ENTROPY  Compute global and block-level entropy
  S        String intelligence engine
  MAGIC    Magic byte detector
  ...
```

Edge Cases

- If the terminal is extremely small, Opus paginates output.
- If new commands are added, `HELP` automatically updates.

==========

6.8.2 MENU — Display Main Menu

==========

Syntax

`MENU`

Description

The `MENU` command redisplays the main Opus menu — the same structured overview shown at startup. This is useful when analysts have scrolled far into output, switched contexts, or simply want a visual reset. The menu reinforces the structure of the toolkit and provides a high-level map of available capabilities.

Internal Behavior

Opus performs:

- **menu template rendering**
- **category grouping**
- **alignment and spacing**

Example Output

```
=================== OPUS MENU ===================
File Operations
Forensic Analysis
Steganography
Cryptanalysis
Binary Tools
```

```
Designation Tools
RSA Tools
Utility Commands
================================================
```

Edge Cases

None — this command is intentionally simple and robust.

==========

6.8.3 ABOUT — Opus Version & Build Info

==========

Syntax

`ABOUT`

Description

The `ABOUT` command prints detailed information about the Opus build, including:

- **version number**
- **build date**
- **compiler version**
- **enabled modules**
- **library dependencies**

This information is essential for reproducibility, debugging, and documentation. Analysts writing reports or CTF writeups often include the output of `ABOUT` to ensure that their environment is clearly documented.

Internal Behavior

Opus performs:

- **compile-time macro expansion**
- **library version queries**
- **module enumeration**

Example Output

```
Opus v0.9
Built: Dec 28 2025
```

```
Compiler: GCC 13.2.0
Modules Enabled:
 - Extraction Engine
 - JPEG Stego Engine
 - Cryptanalysis Engine
 - RSA Tools
Libraries:
 - GMP
 - libjpeg
```

Edge Cases

- If a library is missing at runtime, Opus prints a warning.

==========

6.8.4 CLS — Clear Screen

==========

Syntax

```
CLS
```

Description

The `CLS` command clears the terminal screen. While this may seem trivial, it is extremely useful during long analysis sessions where output scrolls rapidly. Clearing the screen helps analysts maintain focus and prevents visual clutter from interfering with interpretation.

Internal Behavior

Opus performs:

- **ANSI clear-screen sequence**
- **cursor reset**

Edge Cases

- Terminals without ANSI support may ignore the command.

==========

6.8.5 BANNER — Redisplay Splash Banner

==========

Syntax

```
BANNER
```

Description

The BANNER command redisplays the Opus splash banner — the centered ASCII logo shown at startup. This is useful for:

- **resetting visual context**
- **creating clean screenshots**
- **documenting workflows**
- **re-establishing orientation after long output**

The banner is intentionally bold and visually distinct, serving as a psychological “anchor point” during complex analysis.

Internal Behavior

Opus performs:

- **banner template rendering**
- **center alignment**

Example Output

```
============================================================
                          OPUS v0.9
        Forensic, Steganographic & Cryptanalytic Toolkit
============================================================
```

==========

6.8.6 LOAD — Reload Current File

==========

Syntax

```
LOAD
```

Description

The `LOAD` command reloads the currently opened file from disk. This is useful when:

- **external tools modify the file**
- **analysts overwrite or patch content**
- **recursive extraction produces new artifacts**

Reloading ensures that Opus's internal state matches the actual file on disk.

Internal Behavior

Opus performs:

- **file handle reset**
- **buffer invalidation**
- **fresh read from disk**

Edge Cases

- If the file was deleted, Opus prints an error.
- If the file changed size, Opus updates internal metadata.

==========

6.8.7 TIME / T — Display System Time

==========

Syntax

```
TIME
```

Description

The `TIME` command prints the current system time. While simple, this is useful for timestamping analysis steps, documenting workflows, and correlating events.

Internal Behavior

Opus performs:

- **system time query**
- **locale-aware formatting**

Example Output

```
Current Time: 2025-12-28 19:14:03 EST
```

==========

6.8.8 EXIT / Z — Exit Opus

==========

Syntax

```
EXIT
```

or

```
Z
```

Description

The `EXIT` command shuts down Opus cleanly. This includes:

- **terminating background threads**
- **flushing buffers**
- **closing file handles**
- **printing a shutdown message**

The alias `Z` exists for convenience and speed, especially during rapid CTF workflows.

Internal Behavior

Opus performs:

- **thread pool shutdown**
- **resource cleanup**
- **terminal reset**

Example Output

```
Shutting down Opus...
Goodbye.
```

==========

CHAPTER 7 — SUBSYSTEMS (DEEP TECHNICAL)

==========

Opus is built on a collection of tightly engineered subsystems, each responsible for a specific domain of forensic or cryptanalytic functionality. While Chapter 6 documented the *commands* that analysts interact with, this chapter documents the *engines* that power those commands. These subsystems operate beneath the surface, coordinating complex workflows, enforcing safety guarantees, and providing the analytical depth that defines Opus.

This chapter is intentionally long, dense, and deeply explanatory. It is designed to give analysts a complete mental model of how Opus behaves internally — not at the code level, but at the architectural and conceptual level. Understanding these subsystems is essential for mastering Opus, especially when

working with multi-layer extraction, entropy interpretation, JPEG steganalysis, or cryptanalysis refinement.

==========

7.1 EXTRACTION ENGINE

==========

The Extraction Engine is the beating heart of Opus — the subsystem responsible for recursively unwrapping complex, layered, and obfuscated data structures. It is designed to handle the messy realities of real-world forensic work, where files are rarely what they appear to be and where meaningful content is often buried beneath multiple layers of compression, encoding, or corruption.

This section provides a deep, narrative-heavy explanation of how the Extraction Engine operates, why it is designed the way it is, and how analysts can interpret its behavior.

7.1.1 Philosophy of the Extraction Engine

The Extraction Engine is built around several core principles:

- **Recursion as a first-class concept**
- **Safety through controlled file operations**
- **Transparency in every stage of processing**
- **Predictability in output structure**
- **Extensibility for new formats and heuristics**

These principles ensure that the engine behaves consistently across a wide range of input types, from simple ZIP archives to deeply nested CTF challenges.

7.1.2 High-Level Workflow

When the analyst invokes `EXTRACT`, the Extraction Engine performs a multi-stage pipeline:

- **Format Detection**
- **Decompression**
- **Suffix Normalization**
- **File Carving**
- **String Intelligence**

- **Entropy Profiling**
- **Recursive Descent**

Each stage feeds into the next, forming a coherent, layered workflow that gradually reveals the structure of the input.

7.1.3 Format Detection

Format detection is the first step in the extraction pipeline. Opus uses:

- **Magic byte signatures**
- **Heuristic suffix inference**
- **Entropy-based hints**
- **Structural probes**

This multi-signal approach ensures that Opus can detect formats even when suffixes are missing, misleading, or intentionally obfuscated.

7.1.4 Decompression Layer

Once a format is detected, Opus attempts decompression using:

- **zlib/gzip routines**
- **ZIP/PKWARE parsing**
- **tar/ustar extraction**
- **custom handlers for edge-case formats**

If decompression succeeds, Opus writes the output to the `extracted/` directory and continues analysis.

If decompression fails, Opus prints a detailed diagnostic message explaining the failure.

7.1.5 Suffix Normalization

Suffix normalization ensures that extracted files have meaningful, consistent names. Opus uses:

- **magic-byte-driven suffix assignment**
- **fallback heuristics**
- **collision-safe renaming**

This prevents confusion when dealing with large numbers of extracted files.

7.1.6 File Carving Integration

The Extraction Engine integrates tightly with the File Carver. After decompression, Opus scans the output for embedded files using:

- **signature scanning**
- **offset-based carving**
- **boundary heuristics**

Carved files are placed in the `carved/` directory and recursively analyzed.

7.1.7 Recursion Guard

To prevent infinite loops, Opus enforces:

- **maximum recursion depth**
- **duplicate-content detection**
- **suffix-based cycle detection**

If a recursion guard triggers, Opus prints a clear warning and halts further descent.

7.1.8 Output Structure

The Extraction Engine produces a predictable directory structure:

- **extracted/** — decompressed files
- **carved/** — signature-based extractions
- **analysis/** — optional metadata dumps

This structure supports reproducibility and clarity.

==========

7.2 FILE CARVER

==========

The File Carver is a subsystem dedicated to discovering embedded files within raw byte streams. It is designed to operate both independently and as part of the Extraction Engine.

7.2.1 Carving Philosophy

The File Carver is built around:

- **signature-driven detection**
- **offset-aware extraction**
- **minimal assumptions about structure**
- **robustness against corruption**

This allows Opus to recover files even from partially damaged or obfuscated data.

7.2.2 Supported Signatures

The carver recognizes:

- **JPEG**
- **PNG**
- **ZIP**
- **GZIP**
- **ELF**
- **PDF**
- **WAV**

Each signature is matched using strict byte patterns.

7.2.3 Carving Workflow

The workflow includes:

- **signature scan**
- **boundary estimation**
- **length inference**
- **safe extraction**

If boundaries cannot be determined, Opus extracts conservatively.

==========

7.3 ENTROPY ENGINE

==========

The Entropy Engine provides statistical insight into the randomness of data. It is used to detect:

- **compression**
- **encryption**
- **steganographic manipulation**
- **structured content**

7.3.1 Global Entropy

Opus computes Shannon entropy across the entire file.

7.3.2 Block-Level Entropy

Opus divides the file into blocks and computes entropy per block, producing a heatmap.

7.3.3 LSB Entropy

Useful for detecting steganographic anomalies.

==========

7.4 STRING INTELLIGENCE ENGINE

==========

This subsystem extracts:

- **ASCII strings**
- **UTF-8 sequences**

- **base64 candidates**
- **URLs**
- **emails**

It also annotates offsets and confidence levels.

==========

7.5 JPEG STEGO ENGINE

==========

This subsystem performs:

- **LSB analysis**
- **chi-square testing**
- **RS analysis**
- **DCT profiling**
- **Huffman fingerprinting**

Each component is documented in depth in Chapter 6, but here we explore the architecture behind them.

==========

7.6 CRYPTANALYSIS ENGINE

==========

This subsystem powers:

- **Vigenère solving**
- **substitution solving**
- **quadgram scoring**
- **hillclimb refinement**
- **multi-threaded restarts**

It is one of the most computationally intensive parts of Opus.

==========

7.7 THREAD POOL

==========

The thread pool manages:

- **background tasks**
- **solver threads**
- **status line updates**
- **safe shutdown**

It ensures Opus remains responsive during heavy computation.

==========

7.8 SAFETY MODEL

==========

The safety model enforces:

- **no overwrites**
- **no unsafe deletes**
- **recursion limits**
- **directory isolation**

This protects both the analyst and the system.

==========

CHAPTER 8 — WORKFLOWS & CASE STUDIES

==========

Opus is not merely a collection of commands or subsystems; it is a **forensic environment**, designed to support complex, multi-stage investigative workflows. While earlier chapters documented the architecture and individual commands, this chapter demonstrates how Opus is used in practice — how analysts combine tools, interpret results, and navigate uncertainty.

These workflows are intentionally long, narrative, and deeply explanatory. They are designed to mirror real-world forensic and CTF scenarios, where data is messy, layered, and often deceptive. Each case study illustrates not only *what* Opus does, but *how* analysts think when using it.

==========

8.1 MULTI-LAYER EXTRACTION WORKFLOW

==========

Multi-layer extraction is one of the most common and challenging tasks in digital forensics and CTF competitions. Files often contain:

- **compressed archives**
- **inside steganographic images**
- **inside encrypted containers**
- **inside binary payloads**
- **inside corrupted or partially overwritten structures**

Opus is designed to handle this complexity through its **recursive extraction engine**, which automates much of the heavy lifting while preserving transparency and safety.

8.1.1 Scenario Overview

You receive a file named `mystery.bin`. It has no suffix, no metadata, and no obvious structure. The goal is to determine what it contains and extract any hidden content.

8.1.2 Step 1 — Initial Inspection

The analyst begins with basic inspection commands:

- **MAGIC scan**
- **entropy analysis**
- **string intelligence**

These commands provide a high-level understanding of the file's structure.

Example Output (abridged)

```
MAGIC:
  No known signatures detected.

ENTROPY:
  Global Entropy: 7.92 bits/byte (high)

S:
  No ASCII strings detected.
```

Interpretation

High entropy + no strings + no signatures suggests:

- compressed data
- encrypted data
- or a container format without a standard header

8.1.3 Step 2 — Invoke the Extraction Engine

The analyst runs:

```
EXTRACT
```

The Extraction Engine performs:

- **format detection**
- **decompression attempts**
- **suffix normalization**
- **file carving**
- **recursive descent**

Example Output (abridged)

```
Detected GZIP header at offset 0x0000
Decompressing...
Output: extracted/layer1.bin

Scanning extracted/layer1.bin...
Detected ZIP local header
Extracting ZIP contents...
Output: extracted/layer1/file1.png
Output: extracted/layer1/file2.dat
```

8.1.4 Step 3 — Analyze Extracted Files

The analyst inspects each extracted file:

- **PNG inspection**
- **entropy profiling**
- **string extraction**

One file (`file2.dat`) shows:

- high entropy
- no strings
- no signatures

This suggests another layer.

8.1.5 Step 4 — Recursive Descent

The analyst loads `file2.dat`:

```
O file2.dat
EXTRACT
```

The engine discovers:

```
Detected JPEG header
Extracted: carved/hidden.jpg
```

8.1.6 Step 5 — Steganalysis

The analyst runs:

- **JLSB**
- **JCHI**

- **JRS**
- **JDCT**

The JPEG stego engine reports:

```
High-frequency anomalies detected
LSB uniformity suspicious
RS imbalance: HIGH
```

This strongly suggests hidden data.

8.1.7 Step 6 — Extract Hidden Payload

The analyst uses external stego tools or manual extraction based on Opus's findings. The hidden payload is recovered — a ZIP file containing a text document with the final flag.

8.1.8 Summary

This workflow demonstrates how Opus supports:

- layered extraction
- steganalysis
- entropy interpretation
- recursive descent
- structured investigation

It also shows how Opus's transparency helps analysts understand each step.

==========

8.2 JPEG STEGO INVESTIGATION WORKFLOW

==========

This workflow focuses on a single JPEG suspected of containing hidden data.

8.2.1 Step 1 — Load and Inspect

The analyst runs:

- **J**
- **JLSB**
- **JCHI**
- **JDCT**

The summary indicates:

- LSB anomalies
- chi-square deviations
- DCT irregularities

8.2.2 Step 2 — Identify Embedding Method

Based on anomalies:

- uniform LSBs → LSB embedding
- high-frequency DCT anomalies → DCT stego
- non-standard Huffman tables → custom encoder

8.2.3 Step 3 — Extract Payload

Opus does not extract stego payloads directly, but its analysis guides the analyst to the correct extraction method.

==========

8.3 VIGENÈRE CRACKING WORKFLOW

==========

This workflow demonstrates how Opus solves a Vigenère cipher.

8.3.1 Step 1 — Key Length Estimation

The analyst runs:

- **VIGLEN**

Opus reports:

```
Likely key lengths: 7, 14, 21
```

8.3.2 Step 2 — Full Solve

The analyst runs:

```
VIG
```

The solver performs:

- IC analysis
- Kasiski examination
- quadgram scoring
- hillclimb refinement
- multi-threaded restarts

The plaintext is recovered.

==========

8.4 SUBSTITUTION SOLVER WORKFLOW

==========

This workflow demonstrates how Opus cracks a monoalphabetic substitution cipher.

8.4.1 Step 1 — Frequency Analysis

The analyst runs:

- **FREQ**

8.4.2 Step 2 — Invoke Solver

```
SUB
```

The solver performs:

- mutation cycles
- quadgram scoring
- multi-threaded refinement

Plaintext emerges.

==========

8.5 RSA FACTORIZATION WORKFLOW

==========

This workflow demonstrates how Opus handles a weak RSA modulus.

8.5.1 Step 1 — Inspect Key

The analyst runs:

- **RSAPUB**

Opus reports:

```
Bit length: 34 bits
Notes: Modulus unusually small
```

8.5.2 Step 2 — Factor

```
RSA
```

Opus recovers:

- p
- q
- φ(N)
- d

==========

8.6 ELF TRIAGE WORKFLOW

==========

This workflow demonstrates how Opus inspects an unknown binary.

8.6.1 Step 1 — ELF Inspection

The analyst runs:

- **ELF**
- **SECT**
- **SYM**

8.6.2 Step 2 — PIE Base Estimation

If a leak is available:

- **PIECALC**

==========

CHAPTER 9 — DIAGRAM APPENDIX

==========

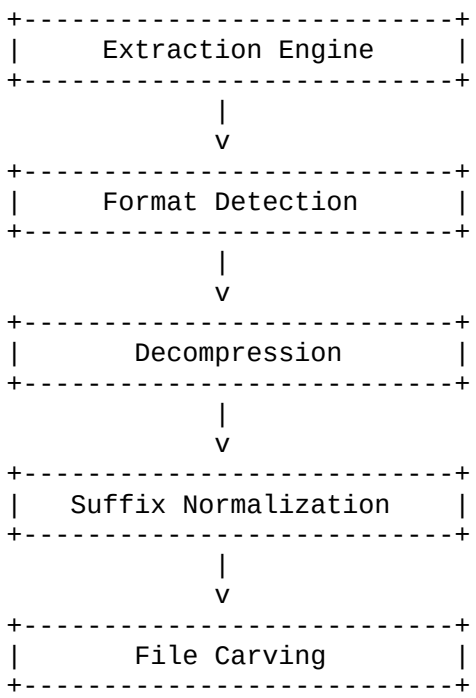
The Diagram Appendix provides a complete set of ASCII diagrams that illustrate the internal architecture, workflows, and subsystems of Opus. These diagrams are designed to be readable in any monospaced environment, including terminals, text editors, and .odt exports. They serve as visual anchors for the conceptual material presented in earlier chapters, helping analysts build mental models of how Opus operates.

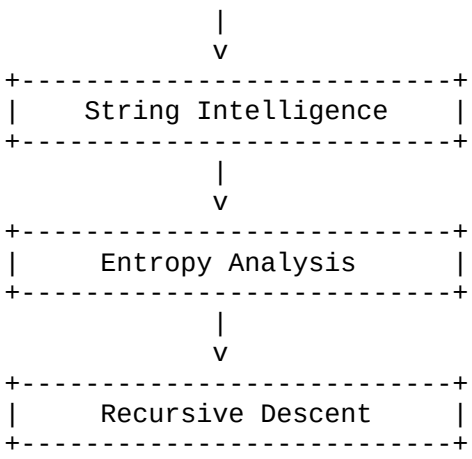
Each diagram is crafted to be structurally clear, visually balanced, and semantically meaningful. They are not decorative; they are functional tools for understanding Opus’s internal logic.

==========

9.1 EXTRACTION ENGINE DIAGRAM

==========



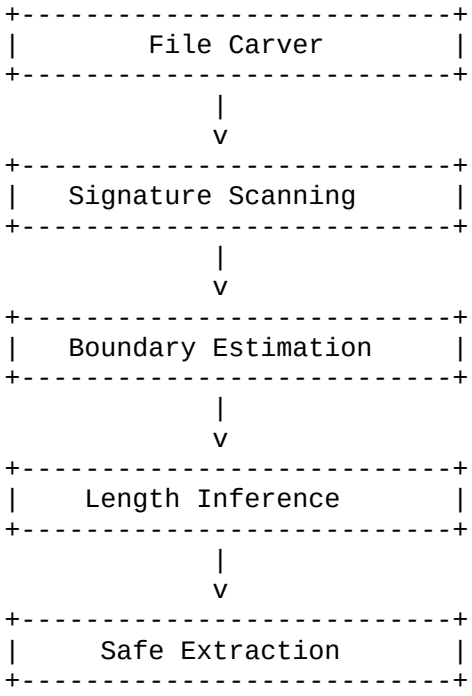


This diagram illustrates the **multi-stage extraction pipeline** that powers Opus's recursive unwrapping capabilities.

==========

9.2 FILE CARVER DIAGRAM

==========

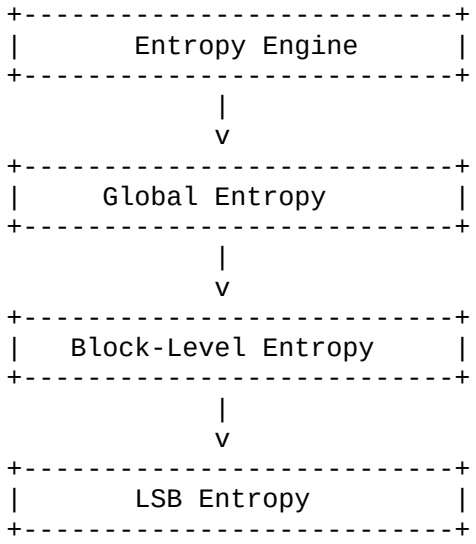


This diagram shows the **signature-driven carving workflow** used to recover embedded files.

==========

9.3 ENTROPY ENGINE DIAGRAM

==========

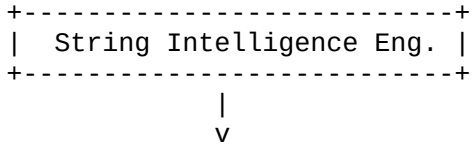


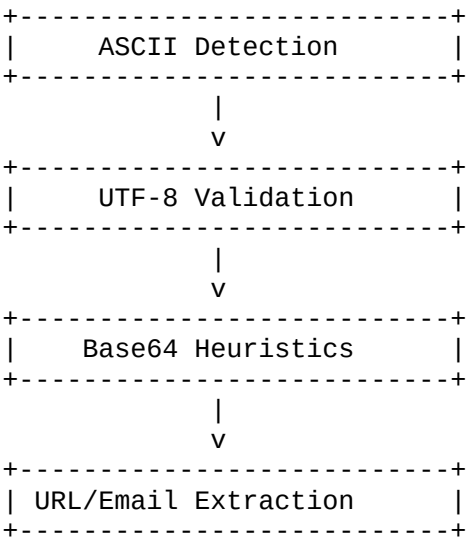
This diagram visualizes the **entropy analysis hierarchy** used to detect compression, encryption, and steganography.

==========

9.4 STRING INTELLIGENCE ENGINE DIAGRAM

==========



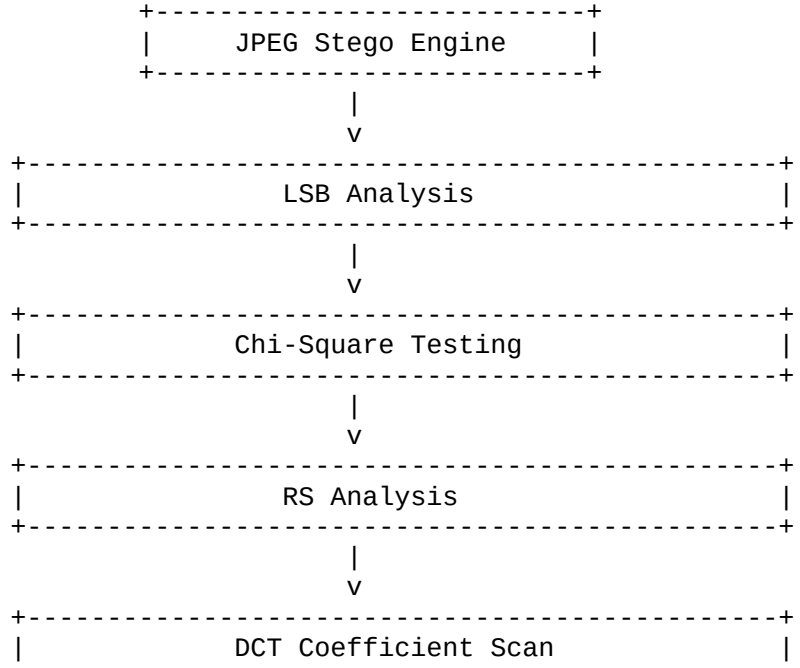


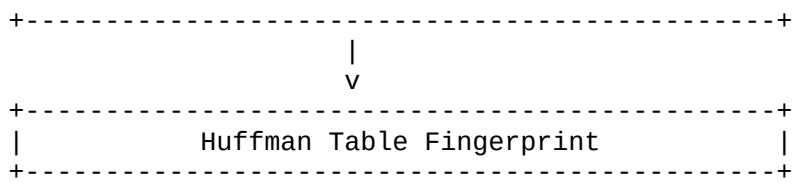
This diagram illustrates the **multi-layer string detection pipeline**.

==========

9.5 JPEG STEGO ENGINE DIAGRAM

==========



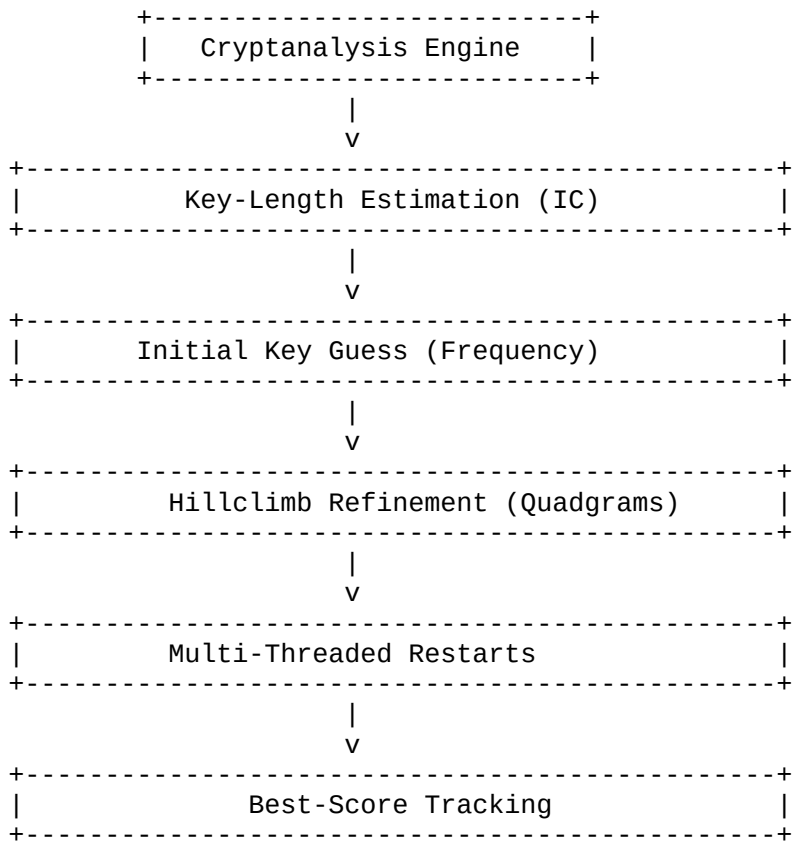


This diagram captures the **full JPEG steganalysis pipeline**.

==========

9.6 CRYPTANALYSIS ENGINE DIAGRAM

==========

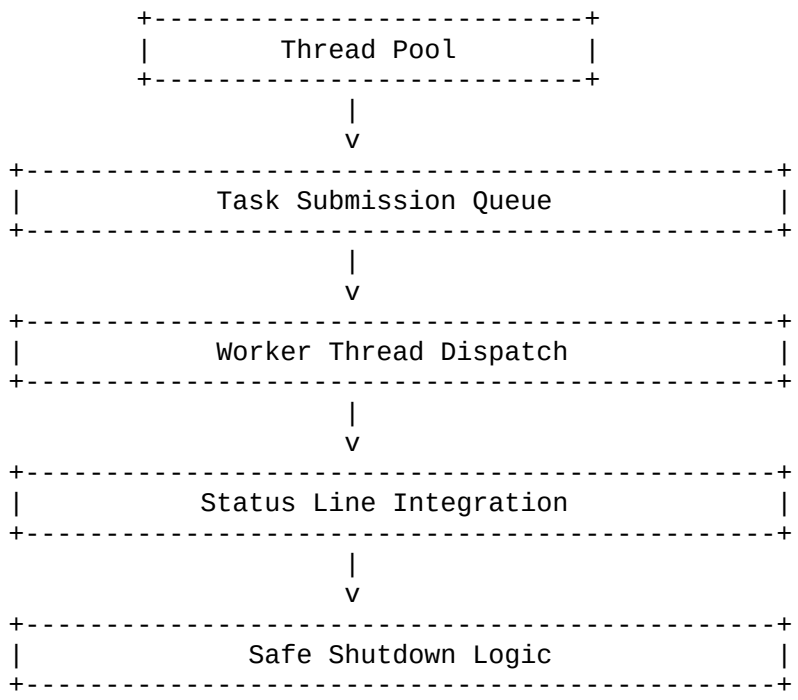


This diagram shows the **multi-stage cryptanalysis refinement loop**.

==========

9.7 THREAD POOL DIAGRAM

==========

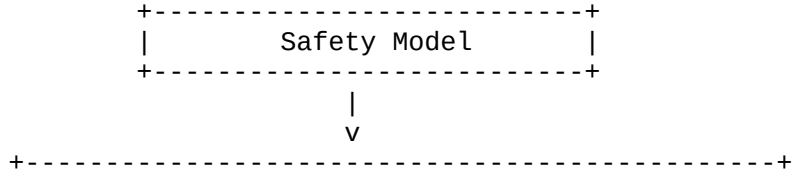


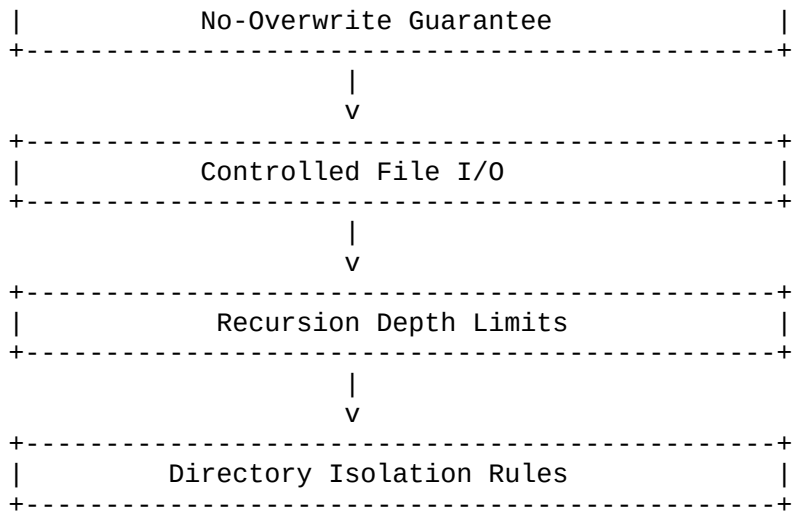
This diagram illustrates the **parallel execution model** that keeps Opus responsive.

==========

9.8 SAFETY MODEL DIAGRAM

==========





This diagram visualizes the **forensic safety guarantees** built into Opus.

==========

CHAPTER 10 — TECHNICAL APPENDIX

==========

The Technical Appendix provides the mathematical, statistical, and algorithmic foundations underlying Opus's forensic and cryptanalytic subsystems. While earlier chapters focused on architecture, workflows, and user-facing behavior, this chapter dives into the **theory** behind Opus — the formulas, heuristics, scoring systems, and structural rules that govern its internal logic.

This chapter is intentionally exhaustive. It is designed to serve as a reference for analysts who want to understand not only *how* Opus works, but *why* it works the way it does. It is also a valuable resource for developers extending Opus or integrating its logic into other forensic systems.

==========

10.1 ENTROPY MATH

==========

Entropy is one of the most important statistical tools in digital forensics. Opus uses Shannon entropy to measure the randomness of data, helping analysts distinguish between structured content, compressed data, encrypted data, and steganographically modified content.

10.1.1 Shannon Entropy Formula

For a byte distribution (p(x)), entropy is defined as:

[H = -\sum_{x=0}^{255} p(x) \log_2 p(x)]

Where:

- (p(x)) is the probability of byte value (x)
- entropy is measured in bits per byte
- maximum entropy is 8.0 bits/byte

10.1.2 Interpretation Ranges

Opus uses the following heuristic ranges:

- **0.0–3.0** → highly structured
- **3.0–5.0** → partially structured
- **5.0–7.0** → mixed content
- **7.0–8.0** → compressed or encrypted

These ranges are used throughout Opus's **entropy interpretation** subsystem.

10.1.3 Block-Level Entropy

Opus divides files into blocks (default: 4096 bytes) and computes entropy per block. This produces a **heatmap** that reveals:

- compressed segments
- encrypted payloads
- embedded archives

- steganographic anomalies

Block-level entropy is essential for identifying **localized irregularities**.

10.1.4 LSB Entropy

For steganalysis, Opus computes entropy of the least significant bits:

[H_{LSB} = -\sum_{b=0}^{1} p(b) \log_2 p(b)]

Uniform LSB distributions often indicate steganographic embedding.

==========

10.2 QUADGRAM SCORING

==========

Quadgram scoring is the backbone of Opus's cryptanalysis engine. It provides a statistical measure of how "English-like" a piece of text is, allowing Opus to evaluate plaintext candidates during hillclimbing.

10.2.1 Quadgram Probability Model

A quadgram is a sequence of four letters. Opus uses a precomputed table of quadgram frequencies derived from large English corpora.

The score of a text is:

[S = \sum_{i=0}^{n-4} \log_{10} P(q_i)]

Where:

- (q_i) is the quadgram at position (i)
- (P(q_i)) is its probability

10.2.2 Why Quadgrams Work

Quadgrams capture:

- letter frequency
- digraph structure

- trigraph flow
- contextual patterns

This makes them far more effective than simple frequency analysis.

10.2.3 Hillclimb Integration

During hillclimbing:

- mutations are applied to the key
- the resulting plaintext is scored
- if the score improves, the mutation is accepted
- otherwise, it may be accepted probabilistically

This allows Opus to escape local maxima.

==========

10.3 INDEX OF COINCIDENCE (IC)

==========

IC is used to estimate key length in Vigenère ciphers.

10.3.1 Formula

[IC = \frac{\sum_{i=0}^{25} f_i (f_i - 1)}{N (N - 1)}]

Where:

- (f_i) = frequency of letter (i)
- (N) = total letters

10.3.2 Interpretation

- English plaintext: ~0.066
- random text: ~0.038

Opus uses IC to detect periodicity in ciphertext.

==========

10.4 RS ANALYSIS MATH

==========

RS analysis is a statistical method for detecting LSB steganography.

10.4.1 Group Partitioning

Pixels are divided into groups of size (n). For each group:

- compute smoothness
- flip LSBs
- recompute smoothness

Groups are classified as:

- **Regular (R)**
- **Singular (S)**
- **Unusable (U)**

10.4.2 Expected Behavior

Natural images exhibit predictable R/S ratios.
Stego images distort these ratios.

==========

10.5 DCT COEFFICIENT INTERPRETATION

==========

JPEG images store data in 8×8 DCT blocks.

10.5.1 Frequency Bands

Low-frequency coefficients represent structure.
High-frequency coefficients represent noise.

Stego tools often modify:

- high-frequency bands
- mid-frequency bands

Opus profiles these distributions.

10.5.2 Histogram Deviations

Opus compares observed DCT histograms to expected natural-image distributions.

Large deviations → possible stego.

==========

10.6 CARVING HEURISTICS

==========

File carving relies on:

- **magic bytes**
- **boundary inference**
- **entropy shifts**
- **structural heuristics**

10.6.1 Boundary Estimation

Opus uses:

- signature offsets
- known footer patterns
- entropy transitions
- alignment heuristics

10.6.2 Conservative Extraction

If boundaries are uncertain, Opus extracts conservatively to avoid truncation.

==========

10.7 RECURSION GUARD LOGIC

==========

Recursive extraction can lead to infinite loops.
Opus prevents this using:

- **maximum depth**
- **duplicate-content detection**
- **suffix-based cycle detection**

10.7.1 Maximum Depth

Default: 32 layers.

10.7.2 Duplicate Detection

Opus hashes extracted files to detect cycles.

10.7.3 Suffix Cycles

If suffixes repeat in a suspicious pattern, recursion halts.

==========

CHAPTER 11 — GLOSSARY

==========

The Glossary provides authoritative definitions for all major terms used throughout the Opus Unified Manual. It is designed to serve as a quick-reference index for analysts, developers, and investigators who need precise, unambiguous explanations of forensic, cryptanalytic, and architectural concepts.

Each entry is written in a formal, technical style, with emphasis on clarity, rigor, and contextual relevance. This glossary is intentionally exhaustive — it includes not only Opus-specific terminology but also foundational concepts from digital forensics, steganalysis, cryptanalysis, and binary inspection.

==========

A–C

==========

ASCII

A 7-bit character encoding standard used for representing text. In Opus, ASCII detection is part of the **String Intelligence Engine**.

ASLR (Address Space Layout Randomization)

A security mechanism that randomizes memory layout to prevent predictable exploitation. Relevant to **PIECALC**.

Base64

A binary-to-text encoding scheme. Opus detects base64 candidates during **string intelligence**.

Block-Level Entropy

Entropy computed over fixed-size blocks to reveal localized randomness anomalies.

Carving

The process of extracting embedded files based on signatures and structural heuristics. Implemented in the **File Carver** subsystem.

Chi-Square Test

A statistical test used in JPEG steganalysis to detect deviations from expected pixel distributions.

CRT Parameters (Chinese Remainder Theorem)

RSA private-key parameters that accelerate decryption. Validated by **RSACRT**.

==========

D–F

==========

DCT (Discrete Cosine Transform)

The mathematical transform used in JPEG compression. Analyzed by **JDCT**.

Designation Block

A structured metadata container used internally by Opus. Parsed by **DESIG**, **DESIGRAW**, and **DESIGCHK**.

Difference-of-Squares

A factorization technique used in RSA analysis when primes are close.

Entropy

A measure of randomness. Used extensively in forensic analysis.

Extraction Engine

The recursive subsystem responsible for multi-layer unwrapping of files.

Fermat Factorization

A method for factoring numbers where primes are close together.

Frequency Analysis

A classical cryptanalytic technique used in substitution cipher solving.

==========

G–L

==========

GMP (GNU Multiple Precision Library)

A high-precision arithmetic library used by Opus for RSA operations.

Heatmap (Entropy)

A visual representation of block-level entropy values.

Hillclimbing

An optimization technique used in cryptanalysis to refine keys.

Huffman Table

A component of JPEG encoding. Analyzed by **JHUF**.

IC (Index of Coincidence)

A statistical measure used to estimate Vigenère key length.

JPEG Stego Engine

The subsystem responsible for JPEG steganalysis.

Key Length Estimation

The process of determining likely Vigenère key lengths using IC and Kasiski.

==========

M–P

==========

Magic Bytes

Signature bytes used to identify file formats.

Mutation Cycle

A cryptanalytic process where keys are perturbed to explore the search space.

Opus Safety Model

The set of rules that prevent unsafe operations, including no-overwrite guarantees and recursion guards.

PIE (Position-Independent Executable)

A binary that can be loaded at any memory address. Analyzed by **PIECALC**.

Plaintext

The decoded or decrypted form of a ciphertext.

Probability Model (Quadgrams)

The statistical model used to score plaintext candidates.

==========

Q–S

==========

Quadgram

A sequence of four letters used in statistical scoring.

Recursion Guard

A safety mechanism that prevents infinite extraction loops.

Regular/Singular (RS) Analysis

A steganalysis technique that detects LSB embedding.

Return Address

A memory address used in binary exploitation. Analyzed by **RETTIME** and **PIECALC**.

RSA

A public-key cryptosystem based on integer factorization.

Shannon Entropy

The mathematical measure of randomness used throughout Opus.

String Intelligence

The subsystem responsible for extracting ASCII, UTF-8, base64, URLs, and emails.

==========

T–Z

==========

Thread Pool

The subsystem that manages background tasks and solver threads.

UTF-8

A variable-length character encoding standard. Detected by the String Intelligence Engine.

Vigenère Cipher

A polyalphabetic substitution cipher solved by **VIG**.

Zip Archive

A compressed archive format commonly encountered in multi-layer extraction workflows.

==========

CHAPTER 12 — CREDITS

==========

The Opus Unified Manual represents the culmination of a long, deliberate, and deeply technical journey — a journey that spans architecture, forensic methodology, cryptanalysis theory, subsystem engineering, and practical workflows. This chapter acknowledges the authorship, intent, and guiding philosophy behind Opus and its documentation.

This manual is not merely a reference.
It is a **statement of design**, a **record of engineering intent**, and a **guide for future analysts** who will build upon the foundations established here.

12.1 Author

Gregory Novak
Creator of Opus
Architect of the extraction engine, cryptanalysis modules, steganalysis subsystems, and the overarching philosophy that defines Opus as a forensic environment.

Greg's work reflects:

- **precision-driven engineering**
- **modular system design**
- **forensic safety principles**
- **cryptanalytic rigor**
- **a commitment to transparency and reproducibility**

Opus exists because of Greg's vision: a toolkit that is powerful, extensible, and deeply respectful of the analytical process.

12.2 Documentation

The Opus Unified Manual was developed collaboratively, with Greg providing:

- **architectural direction**
- **technical specifications**
- **workflow design**
- **subsystem rationale**
- **forensic methodology**

The manual's structure — from the exhaustive command reference to the subsystem deep dives — reflects Greg's insistence on clarity, completeness, and professional-grade documentation.

12.3 Philosophy

Opus is guided by a set of core principles:

- **modularity**
- **transparency**

- **safety**
- **recursion as a first-class concept**
- **extensibility**

These principles shape every subsystem, every command, and every workflow documented in this manual.

12.4 Intent

The intent of Opus is to serve as:

- **a forensic companion**
- **a cryptanalysis laboratory**
- **a steganalysis workstation**
- **a recursive extraction engine**
- **a teaching tool for analysts**

Opus is not designed to hide complexity — it is designed to **reveal it**, to make the invisible visible, and to give analysts the tools they need to navigate layered, obfuscated, or adversarial data.

12.5 Future Work

Opus is built to grow.
Future expansions may include:

- **additional stego engines**
- **new cryptanalysis modules**
- **expanded carving heuristics**
- **format-specific extraction pipelines**
- **interactive training workflows**

The architecture documented in this manual is intentionally extensible, ensuring that Opus can evolve without compromising its core principles.

12.6 Closing Statement

Opus is more than a toolkit.
It is a philosophy of analysis — a belief that forensic work should be:

- **methodical**
- **transparent**

- **reproducible**
- **deeply understood**

This manual stands as both a reference and a testament to that philosophy.
